

PEDA: Privacy-Enhancing Distance-Aware Aggregation of Graph Neural Networks

Yuqing Wang¹, Junwei Zhang¹, Zhuo Ma¹, *Senior Member, IEEE*, Jinhai Zhang¹,
Xinghua Li¹, *Member, IEEE*, and Jianfeng Ma¹, *Member, IEEE*

Abstract—Graph neural networks (GNNs) are extensively employed in location-related scenarios, relying on aggregation to gather features from neighboring nodes based on edge weights. Features are closely bound to nodes' locations and edge weights mirror distance correlations. In this sense, certain privacy concerns exist while providing location-based services if there is insufficient privacy protection. To this end, we propose a privacy-preserving and distance-aware data aggregation framework (PEDA) for GNNs. Specifically, PEDA achieves location privacy by combining circular-based positional coding with inner product functional encryption. Because of the masks in the codes, the decryption returns masked distances, preventing distance leakage. Following this, in order to protect feature privacy, we employ secret sharing. To preserve the collection strategy's privacy, we implement an oblivious transfer for collecting the shared features. Additionally, we securely generate the adjacency matrix and aggregate features based on multi-party computation. Thorough security analysis and comprehensive evaluation demonstrate the privacy, feasibility and practicality of our approach. When compared to related works, PEDA offers four types of privacy, maintains distance awareness and feature utility, and allows for oblivious data collecting with little computational cost sacrifice.

Index Terms—Privacy-preserving, distance-aware, secure aggregation, location privacy, graph neural network.

I. INTRODUCTION

IN RECENT years, the rapid development of graph neural networks (GNNs) has injected new innovations into various areas including recommender systems [1] and biomedicine [2], particularly in location-based domains such as transportation [3] and weather forecasting [4]. GNNs succeed at graph-based tasks by relying on aggregation to capture structural information and topological relationships between data sources and exploiting feature dependencies for model training. However, there are significant potential privacy concerns with many components of such aggregation. Thus, in reaction to this, we put out a novel method that guarantees both privacy protection and distance awareness.

Received 11 October 2024; revised 29 August 2025; accepted 3 October 2025. Date of publication 14 October 2025; date of current version 24 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372345, Grant U21A20464, Grant 62125205, Grant 62232013, and Grant U23A20303; and in part by the Natural Science Basic Research Program of Shaanxi under Grant 2023-JC-JQ-49. The associate editor coordinating the review of this article and approving it for publication was Prof. Jianting Ning. (Corresponding author: Junwei Zhang.)

The authors are with the State Key Laboratory of Integrated Service Networks and the School of Cyber Engineering, Xidian University, Xi'an 710071, China (e-mail: wangyuqing@stu.xidian.edu.cn; jwzhang@xidian.edu.cn; mazhuo@mail.xidian.edu.cn; jhzhang_3@stu.xidian.edu.cn; xhli1@mail.xidian.edu.cn; jfma@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3620680

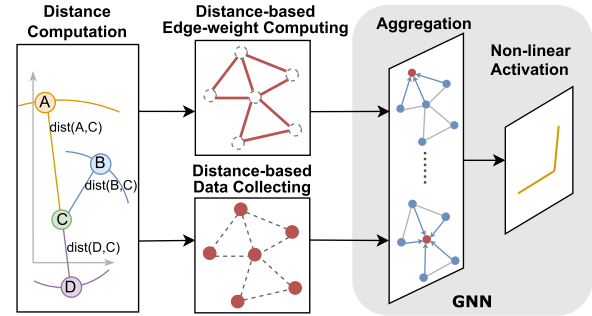


Fig. 1. A distance-aware aggregation framework for GNNs.

As shown in Fig. 1, the *Aggregation* gathers features from neighbor nodes according to edge weights, resulting in new values for the central node. In location-related scenarios, *Distance-based Edge-weight Computing* generates edge weights reflecting positional relationships between nodes. Also, features are collected following the defined distance-aware collecting strategies in *Distance-based Data Collecting*. Both require *Distance Computation*. Among them, there is a substantial possibility of location leakage.

While location privacy and distance awareness may appear incompatible, they are both required in scenarios involving locations. On one hand, location is sensitive information closely related to citizens' privacy, such as temples, hospitals, and companies, which are enough to expose one's religion, health, profession, etc. On the other hand, distance-awareness is a fundamental property of a location-dependent GNN. As an illustration, in multi-site photovoltaic (PV) power forecasting [5], each station corresponds to a node in the graph where the edge weight between two nodes is a measure between two sites such as distance correlation.

At this point, we find that GNNs are highly prone to leaking spatial information. The above modules can pose a threat to location privacy as follows. (1) *Distance Computation*. It is obvious that the process of computing distances involving nodes' locations easily contributes to the location leakage. Besides, distance describes the length between two nodes, reflecting more sensitive information such as the similarity between the two nodes. In this way, distance is equally sensitive. (2) *Distance-based Edge-weight Computing*. Edge weight is a measure of distance correlations, whose leakage causes the exposure of distances between nodes. Therefore, edge weight privacy is also important as well as distance privacy. (3) *Distance-based Data Collecting*. The data-collecting rule is

bound to distances. Data suppliers can draw conclusions from whether their features are collected and infer the collecting strategies, achieving a portion of the spatial information.

The existing work on privacy-preserving GNNs has limitations when dealing with the above privacy issues. On one hand, some scholars [6] use differential privacy (DP) technology to achieve node-level and edge-level privacy. However, the adjacency matrix is still publicly available, thereby the leakage risk is still unavoidable. Additionally, due to DP adding noise to node features, the usability of features is reduced. On the other hand, some methods [7] are based on federated learning, where clients train features locally before aggregating. Such methods are not suitable for scenarios where the entire graph needs to be manipulated [8].

A. Our Contributions

In this paper, in order to tackle the above-mentioned privacy concerns, we propose PEDa, a privacy-preserving and distance-aware data aggregation framework for GNNs. Firstly, private distances between nodes are computed without leakage of locations. Then, features are shared and collected obliviously, and an adjacency matrix is generated securely, both of which contribute to a secure aggregation. We also analyze its security and evaluate its performance.

- **Distance aware and location privacy.** We give a private distance computation method with location privacy, creatively integrating positional coding with inner product functional encryption. Specifically, locations are encoded into vectors and encrypted under a functional encryption. The decryption is the inner product of two vectors, which in turn obtains the distance. Therefore, without losing location privacy, PEDa realizes distance-awareness.
- **Privacy preserving adjacency matrix generation.** The adjacency matrix in the location-related GNN corresponds to sensitive distance correlations. We provide a novel and secure method for adjacency matrix generation based on multi-party computation (MPC) with input shared squared distances, ensuring both distance privacy and adjacency matrix privacy.
- **Oblivious data collecting.** Sensitive features are gathered following collecting strategies based on masked distances. We introduce oblivious transfer (OT) so that data suppliers can not be aware of whether their features would be aggregated. In other words, OT guarantees that data suppliers may infer no information about the server's collecting strategy. In addition, feature vectors are additively secret-shared to ensure feature privacy.

II. PRELIMINARIES

A. Inner Product Function Encryption

Two-client Inner-Product Functional Encryption (IPFE) [9] enables two parties to independently encrypt vectors x and y , so that a third party can derive a functional decryption key to compute xy^T . An IPFE scheme comprises four algorithms:

- **Setup(1^k).** The groups G_1, G_2, G_T , the generators g_1, g_2, g_t and p are set to be public parameters.

$B = (b_i)_i \xleftarrow{\$} GL_{12}(\mathbb{Z}_p)$ and $B^* = (b_i^*)_i \leftarrow (B^{-1})^T$ are random dual orthogonal matrices, which define the bases $(\mathbb{B}, \mathbb{B}^*)$. Sample $Q = (\delta_i)_i, M = (\mu_i)_i \xleftarrow{\$} (\mathbb{Z}_p^3)^5$. This algorithm generates the encryption keys ek_1 and ek_2 for DP₁ and DP₂ respectively, and the secret key sk , where $ek_1 = (B, M), ek_2 = (B^*, Q), sk = (M, Q)$.

- **Enc(x).** Given the label l , one encrypts a vector x with an encryption key ek into the ciphertext X .
- **Keygen(sk).** With the secret key $sk = (M, Q)$, one generates the decryption key $k = \sum_{\rho=1}^5 (\mu_{\rho,1}\delta_{\rho,1}, \mu_{\rho,2}\delta_{\rho,2}, \mu_{\rho,3}\delta_{\rho,3}) \in \mathbb{Z}_p^3$.
- **Dec(X, Y, k).** Where X is a ciphertext obtained from DP₁ and Y from DP₂ under labels l_1 and l_2 respectively, returns $xy^T = \sum_i x_i y_i$ if $l_1 = l_2$.

The security properties of IPFE include message privacy, minimal leakage, and IND-CPA security, where message privacy is formalized as follows.

B. IPFE-Message Privacy

Let Π be the inner-product functional encryption. Given ciphertexts $X \leftarrow \text{Enc}(\vec{x}), Y \leftarrow \text{Enc}(\vec{y})$, decryption key $k \leftarrow \text{Keygen}(sk)$ where $sk \leftarrow \text{Setup}(1^k)$, and result $z \leftarrow \text{Dec}(X, Y, k)$, for every Probabilistic Polynomial-Time (PPT) adversary $\mathcal{A}$ there is a negligible function $\text{negl}(\cdot)$ with a sufficiently large λ :

$$\text{Prob}[(\vec{x}, \vec{y}) \leftarrow \mathcal{A}_{\Pi}(X, Y, k, z) : z = \vec{x} \circ \vec{y}] \leq \text{negl}(\lambda).$$

C. Additive Secret Sharing

Secret sharing (SS) is a cryptographic tool that allows a user to share a private value x into shares in such a way that any strict subset of these values reveals nothing about x , but all values together can be used to reconstruct x completely. Given the secret x and a set of users $\mathcal{U}$, the algorithms are $\text{Share}(x) \rightarrow ([x]_1, \dots, [x]_{|\mathcal{U}|})$ and $\text{Recon}([x]_1, \dots, [x]_{|\mathcal{U}|}) \rightarrow x$, where each $[x]_u$ is subject to the constraint $\sum_{u \in \mathcal{U}} [x]_u = x$.

The properties of additive secret sharing include additivity and perfect privacy. Additivity is straightforward while perfect privacy is formally described below.

SS-Perfect privacy [10]. Let Π be an additive secret sharing. x is the secret and $[x]_i$ is P_i 's share. Given n parties and $n - 1$ shares or less, for every PPT adversary $\mathcal{A}$ who corrupt $P_i (i \neq j)$, there is a $\text{negl}(\cdot)$ with a sufficiently large λ :

$$\text{Prob}([x], [x]_j) \leftarrow \mathcal{A}_{\Pi}([x]_i, n) : \{[x]_1, \dots, [x]_n\} \leftarrow \text{Share}(x); x \leftarrow \text{Recon}(\{[x]_1, \dots, [x]_n\}) \leq \text{negl}(\lambda).$$

D. Oblivious Transfer

In Oblivious Transfer (OT) [11], a sender with two inputs (m_0, m_1) interacts with a receiver who has an input choice bit $b \in \{0, 1\}$. The result is that the receiver learns m_b while the sender learns nothing about b . OT ensures that the sender does not learn which bit the receiver has chosen; the receiver knows nothing about m_{1-b} , the value that he did not choose.

OT ensures both sender and receiver privacy. We focus on receiver privacy, which is formalized as follows. OT-Receiver security [12]. Let π be an oblivious transfer. Given m_0, m_1 , for

TABLE I

BASE MULTI-PARTY COMPUTATION (MPC) PROTOCOLS

Protocols	Inputs	Outputs	Functionality
$\pi_{Compare}$ [13]	X, Y	$[1\{X < Y\}]_b^B$	$1\{X < Y\}$
π_{Sqrt} [14]	$[X]_b$	$[Z]_b$	$Z = \sqrt{X}$
π_{MatMul} [15]	$([X]_b, [Y]_b)$	$[Z]_b$	$Z = XY$
$\pi_{EWMatMul}$ [16]	$([X]_b, [Y]_b)$	$[Z]_b$	$Z = X \otimes Y$

$\otimes$ symbol representing the element wise multiplication.

every PPT adversary $\mathcal{A}$ who corrupts the sender, there is a negligible function $negl(\cdot)$ such that for a sufficiently large λ :

$$Prob[b \leftarrow \mathcal{A}_\Pi(m_0, m_1)] \leq \frac{1}{2} + negl(\lambda).$$

E. MPC Building Blocks

We utilize some multi-party computation protocols in TABLE I. $\pi_{Compare}$ [13] returns a comparison result between two input values without leaking any information. π_{Sqrt} [14] is used to compute the square root of a value by simply inputting its additive shares. Similarly, based on the additively shared values, π_{MatMul} can calculate the matrix multiplication of two matrices. $\pi_{EWMatMul}$ is a variant of π_{MatMul} , which is a protocol of element-wise matrix multiplication. MPC-Input privacy. Let Π be an MPC protocol [17] related to functionality f , containing n non-collusive parties ($n \geq 2$) in a semi-honest model. x_i, y_i are shares of x, y . For every PPT adversary $\mathcal{A}$ who corrupt P_i ($i \neq j$), there is a negligible function $negl(\cdot)$ such that for a sufficiently large λ :

$$Prob[(x_j, y_j) \leftarrow \mathcal{A}_\Pi(x_i, y_i, n) : y = f(x)] \leq negl(\lambda).$$

F. Graph Neural Networks

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, A)$ be a graph with n nodes $\mathcal{V}$, edges $\mathcal{E}$. An adjacency matrix $A \in \mathbb{R}^{n \times n}$ encoding the edge weight between two nodes describes the graph structure. A feature description X_i for every node is summarized in an $n \times d$ feature matrix X where d is the number of input features. The goal of modeling GNNs is to use a graph structure to describe the information aggregation of neighboring nodes [18].

The majority of GNN models share a largely universal architecture. The transformation in every neural network layer can then be written as follows:

$$X^{(l+1)} = upd(agg(X^{(l)}, A)), \quad (1)$$

where $X^{(0)} = X$. The $upd(\cdot)$ may be $ReLU(\cdot)$ or $\sigma(\cdot)$. The specific models then differ only in how the aggregation function $agg(\cdot)$ is chosen and parameterized. $agg(\cdot)$ is used to learn spatial features and obtain spatial dependence.

1) *A Simplified and Universal Aggregation*: To aggregate features of neighboring nodes based on a weighted adjacency matrix for a certain node, the multiplication of A and X is equivalent to the addition of neighboring nodes' features as

$$agg(X, A) = AX. \quad (2)$$

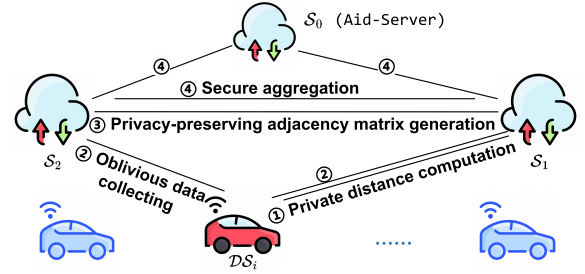


Fig. 2. System model of PEDAs.

2) *An Improved Aggregation*: 2-layer GCN model sums normalized features from all one-hop of neighboring nodes [19] but compensates for shortcomings of the simplified aggregation described above. We focus on the aggregation module:

$$agg(X^{(l)}, A) = \hat{A}X^{(l)}, \quad (3)$$

where $\hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$, $\tilde{A} = A + I_N$ is the matrix with added self-connections. I_N is the identity matrix. $\tilde{D}$ is the degree matrix, $\tilde{D} = \sum_j \tilde{A}_{ij}$.

III. PROBLEM FORMULATION

A. System Model

As illustrated in Fig. 2, we introduce the following entities.

1) *Data Suppliers*: $\{DS_i\}_{i \in \mathcal{U}_1}$. There is a set of data suppliers $\mathcal{U}_1$. Each DS is considered as a node in the graph. DS s firstly encode and encrypt their locations. Then, they send ciphertexts to the server S_1 for computing distance-related values. Furthermore, DS s can provide servers with sensitive features in a privacy-preserving manner, ensuring that the underlying data remains confidential.

2) *Servers*: S_0, S_1 and S_2 . S_1 decrypts the ciphertexts from DS s to obtain masked distances, which remain protected by random masks, thereby preventing leakage of true distances. S_1 and S_2 collect sensitive features from DS s and form the feature matrix for aggregation. Then, they jointly generate an adjacency matrix privately. S_0 is an aid server in secure aggregation, helping S_1 and S_2 to perform secure computations.

PEDA works as follows in brief. In the beginning, all DS s encode locations and random masks into vectors, then encrypt vectors and send ciphertexts of vectors to S_1 . Next, S_1 performs decryption on ciphertexts and gets the inner products of vectors, which are the masked squared distances. This can be regarded as secret sharing between the two parties as the masks held by DS s. After that, S_1, S_2 , and DS s comply with the collecting rule, and shared features are collected from reachable data suppliers obliviously. Furthermore, S_1 and S_2 construct the adjacency matrix from the shared squared distances by applying the threshold Gaussian kernel, while ensuring that the actual distances are not revealed. Finally, servers perform secure aggregation with the help of multi-party computation modules.

B. Threat Model

We assume that data suppliers and servers are honest-but-curious, meaning they strictly follow the prescribed protocol but may attempt to infer additional information.

- $\mathcal{S}_1$ has the incentive to use collected messages to track $\mathcal{DS}$ s' precise **locations**, which is highly sensitive information for $\mathcal{DS}$ s, revealing personal behaviors.
- Moreover, $\mathcal{S}_1$ may conduct **multiple queries** and exploit the corresponding responses to infer locations.
- $\mathcal{DS}$ s, $\mathcal{S}_1$ and $\mathcal{S}_2$ intend to obtain the **distances** with potential malicious purposes, aiming to infer sensitive information or gain unauthorized advantages.
- Servers can use collected messages to infer sensitive information about **features** and the **adjacency matrix**. The personal information of $\mathcal{DS}$ s is easy to compromise, assuming features leak. The adjacency matrix reflects the edge weights between $\mathcal{DS}$ s. Such data might be marketed to other third parties, including advertising companies, or utilized to enhance their services.
- $\mathcal{DS}$ s have the incentive to deduce the **data collecting strategy**. $\mathcal{DS}$ s infer whether their features have been accepted, thereby potentially inferring location preference or other sensitive information of servers.

We further assume that no collusion occurs between any two parties. This non-collusion assumption is realistic. For example, the city traffic authority ($\mathcal{S}_1$) and a commercial service provider ($\mathcal{S}_2$) can jointly compute traffic models without accessing raw data. $\mathcal{S}_1$ is subject to strict regulatory oversight, while $\mathcal{S}_2$ relies on its commercial reputation and long-term business value to maintain market competitiveness; any attempt to reconstruct raw data would pose severe legal and reputational risks, providing strong disincentives for collusion. Moreover, data suppliers are unlikely to collude with servers due to potential conflicts of interest, further supporting the validity of the non-collusion assumption.

IV. PROPOSED PEDAs

A. Overview

We introduce the architecture of PEDAs shown in Fig. 3.

- ① Firstly, to calculate distances securely, we design a positional coding and combine it with IPFE. Accordingly, the distance can be decrypted from IPFE ciphertexts of vectors, provided locations have been encoded to vectors. Moreover, a random value is introduced in the vector to mask the distance. Thus, the mask and the masked distance are the two additive shares of a squared distance.
- ② Next, inputting such shares, oblivious transfer and additive secret sharing are implemented to obtain sensitive features. We define a free-flow reachable collecting rule and compute the distance-related collecting strategies securely. In addition, feature vectors are split by secret sharing to maintain privacy. Then, the shared feature vectors are obliviously collected using OT to create an additive shared feature matrix, $[X]_1 + [X]_2 = X$.
- ③ Then, the servers construct the adjacency matrix A based on a threshold Gaussian kernel, which requires the variance of all distances, $\sigma^2 = E(dist^2) - E(dist)^2$. To compute it without leaking distance information, 2PC sub-protocols are introduced to securely compute both $E(dist^2)$ and $E(dist)$. Two servers then generate

TABLE II
LIST OF NOTATIONS

Notation	Description
G_1, G_2, G_T	Group with the generators g_1, g_2, g_t
B, B^*	Random dual orthogonal matrices defining bases($\mathbb{B}, \mathbb{B}^*$)
Cek, Nek, dk	IPFE encryption keys, decryption key
$\overrightarrow{CN_i}$	Code of central node $\mathcal{DS}_i$
$\overrightarrow{NN_{i,j}}$	Code of neighbor node $\mathcal{DS}_j$ when $\mathcal{DS}_i$ is the center
$dist_{i,j}$	Distance from $\mathcal{DS}_i$ to $\mathcal{DS}_j$
$r_{i,j}$	A randomly sampled mask used to mask $dist_{i,j}$
$s_{i,j}$	The masked squared distance, s.t. $s_{i,j} = dist_{i,j}^2 - r_{i,j}$
$FFR_{i,j}$	Free-Flow reachability from $\mathcal{DS}_i$ to $\mathcal{DS}_j$
CR_i	Collecting strategy for $\mathcal{DS}_i$
$\{m_{i,0}^b, m_{i,1}^b\}$	messages of a sender $\mathcal{DS}_i$ when the receiver is $\mathcal{S}_b$
X_i	The feature vector of $\mathcal{DS}_i$, $X_i \in R^{1 \times P}$
X	Feature matrix, $X \in R^{ \mathcal{U}_1 \times P}$
A	Adjacency matrix, $A \in R^{ \mathcal{U}_1 \times \mathcal{U}_1 }$
CAI, CAz	Element-wise multiplication factors, $CAI \otimes CAz = A$
σ	Standard deviation of all distances
$E(\cdot)$	Mathematical expectation
$[\cdot]_b$	An additive secret share for $\mathcal{S}_b$

intermediate matrices whose element-wise multiplication produces the adjacency matrix A .

- ④ Finally, we transfer two classic aggregation functions described in Section II-F to secure ones. Secure aggregation is completed based on MPC sub-protocols to ensure the privacy of the feature matrix and adjacency matrix.

The notions and descriptions are listed in TABLE II.

B. Detailed Construction

The groups G_1, G_2, G_T , the generators g_1, g_2, g_t and p are public parameters. For $i \in \mathcal{U}_1$, honestly generate $(Cek_i, Nek_i, sk_i) \leftarrow \text{IPFE.Setup}(1^k)$, where the encryption key pair $Cek_i = (B, M_i)$, $Nek_i = (B^*, Q_i)$, the secret key $sk_i = (M_i, Q_i)$. Cek_i is sent to $\mathcal{DS}_i$ while Nek_i is sent to $\mathcal{DS}_j$. sk_i is sent to $\mathcal{S}_1$. Then, $\mathcal{S}_1$ generates the decryption key dk_i from sk_i as $dk_i = \sum_{\rho=1}^5 (\mu_{\rho,1}\delta_{\rho,1}, \mu_{\rho,2}\delta_{\rho,2}, \mu_{\rho,3}\delta_{\rho,3}) \in \mathbb{Z}_p^3$. $\{l_{i,j}\}_{j \neq i, i \in \mathcal{U}_1, j \in \mathcal{U}_1}$ are labels. In this way, $\mathcal{S}_1$ holds $\{dk_i\}_{i \in \mathcal{U}_1}$, $\mathcal{DS}_i$ holds Cek_i and $\{Nek_j\}_{j \neq i, j \in \mathcal{U}_1}$.

① Private distance computation

Before detailing the algorithms, a simple example demonstrates positional encoding and encryption, shown as Fig. 4. Fig. 5 further illustrates interactions and encrypted distance calculation. Consider two nodes located at coordinates $(x_c, y_c) = (1, 2)$ and $(x_n, y_n) = (4, 3)$. By applying positional encoding and encryption using keys (Cek, Nek) , computations can be performed directly on the ciphertexts. Subsequent decryption with dk yields the distance-related value $\overrightarrow{CN} \circ \overrightarrow{NN} = (1, 1, 2, 5, -1) \circ (25, -8, -6, 1, r) = 10 - r$, where 10 is the squared distance and r is a random mask. Thus, the masked squared distance can be obtained directly from the ciphertexts without revealing the true locations.

1) *Positional encoding*: Inspired by Wang et al. [20], which transformed the circle equation $(x-x_c)^2 + (y-y_c)^2 - R^2 = 0$ into an inner product form $(x^2 + y^2, -2x, -2y, 1) \circ (1, x_c, y_c, x_c^2 + y_c^2 - R^2) = 0$, we design a positional encoding to convert distance computations into inner product operations.

Consider that locations are mapped onto a 2D coordinate system, which may be based on longitude-latitude or cus-

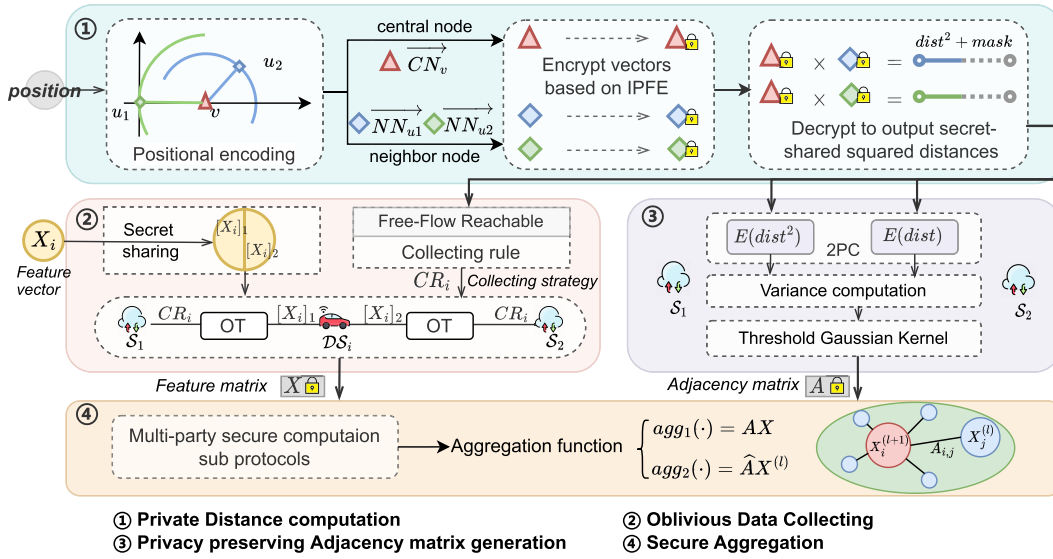


Fig. 3. The architecture of PEDAs.

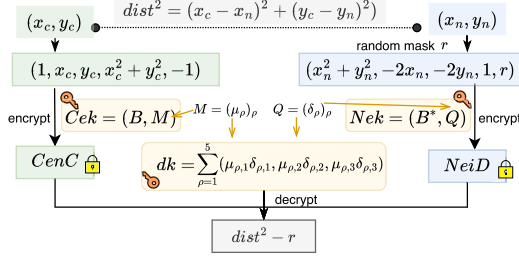


Fig. 4. An illustrative example of positional encoding and encryption.

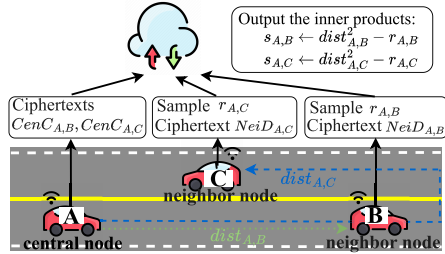


Fig. 5. An example of distance computation based on IPFE.

tomized mapping rules (e.g., road distances where $dist_{i,j} \neq dist_{j,i}$). To support this flexibility, $\mathcal{DS}$ encode nodes in two roles: central node (x_c, y_c) and neighbor node (x_n, y_n) . As described in Protocol 1, the vectors are defined as $\vec{CN} = (1, x_c, y_c, x_c^2 + y_c^2, -1)$ and $\vec{NN} = (x_n^2 + y_n^2, -2x_n, -2y_n, 1, r)$, where a randomly sampled r is used as a mask to protect distance privacy. Given that $\circ$ is an inner product operation, the following equation may be employed:

$$\vec{CN} \circ \vec{NN} = (x_c - x_n)^2 + (y_c - y_n)^2 - r.$$

Thus, distance evaluation is transformed into an inner product with built-in masking. It is not difficult to find that the inner product, $\vec{CN} \circ \vec{NN}$, is a masked squared distance.

2) **Encryption Phase:** To enable distance awareness while preserving location privacy, $\mathcal{DS}$ s encrypt positional vectors using IPFE. This encryption allows inner products to be computed directly over the resulting ciphertexts without revealing

Protocol 1 Positional Encoding

Input: $\mathcal{DS}_i$'s coordinate point. $x_c, y_c, x_n, y_n \in \mathbb{Z}_p$

Output: Vectors $\vec{CN}_i, \vec{NN}_{j,i}$ for $j \in \mathcal{U}_1, j \neq i$.

- 1: Denote $\mathcal{DS}_i$'s point as (x_c, y_c) where $\mathcal{DS}_i$ acts as a central node.
- 2: Generate $\mathcal{DS}_i$'s central node vector $\vec{CN}_i = (1, x_c, y_c, x_c^2 + y_c^2, -1)$.
- 3: **for** each other $\mathcal{DS}_j (j \in \mathcal{U}_1, j \neq i)$ as a central node **do**
- 4: Denote $\mathcal{DS}_j$ as a neighbor node (x_n, y_n) .
- 5: Sample the mask randomly $r_{j,i} \xleftarrow{\$} \mathbb{Z}_p$.
- 6: $\mathcal{DS}_i$ generates the vector $\vec{NN}_{j,i} = (x_n^2 + y_n^2, -2x_n, -2y_n, 1, r_{j,i})$.
- 7 **end for**

the underlying vectors. During the subsequent decryption phase, these inner products can be recovered using the decryption key dk , representing the masked squared distances.

Protocol 2 details the encryption process. As described in Line 4-9, $\mathcal{DS}_i$ encrypts $\vec{CN}_i$ with encryption key $Cek_i = (B, M_i)$. For each $\mathcal{DS}_j$, with label $l_{i,j}$, the ciphertext $CenC_{i,j}$ is outputted. Then, for each other $\mathcal{DS}_j$, $\mathcal{DS}_i$ encrypts $\vec{NN}_{j,i}$ with encryption key $Nek_j = (B^*, Q_j)$ correspondingly. Similar to the encryption of $\vec{CN}_i$, $\mathcal{DS}_i$ produces the ciphertext $NeiD_{j,i}$. Each ciphertext consists of multiple encrypted components indexed by ρ , where

$$C_\rho = (\pi_\rho, \pi_\rho \lambda_\rho, u_\rho, 0, 0, \sigma_1 \mu_{\rho,1}, \sigma_2 \mu_{\rho,2}, \sigma_3 \mu_{\rho,3}, 0, 0)_{\mathbb{B}},$$

$$D_\rho = (\gamma_\rho \lambda_\rho, -\gamma_\rho, v_\rho, 0, 0, \xi_1 \delta_{\rho,1}, \xi_2 \delta_{\rho,2}, \xi_3 \delta_{\rho,3}, 0, 0)_{\mathbb{B}^*}.$$

The bases $(\mathbb{B}, \mathbb{B}^*)$ are defined from B and B^* , which are random dual orthogonal matrices.

3) **Decryption Phase:** As described in Protocol 3, for each pair $\mathcal{DS}_i$ and $\mathcal{DS}_j$, $\mathcal{S}_1$ can compute the inner product, denoted by $\vec{s}_{i,j}$. More specifically, given decryption key $\vec{d} = \sum_{\rho=1}^5 (\mu_{\rho,1} \delta_{\rho,1}, \mu_{\rho,2} \delta_{\rho,2}, \mu_{\rho,3} \delta_{\rho,3})$, $C_0 \leftarrow \vec{\sigma} \cdot G_1$, $D_0 \leftarrow \vec{\xi} \cdot G_2$, due to the orthogonality of the bases $\mathbb{B}$ and $\mathbb{B}^*$, with

Protocol 2 Encrypt the Vectors Into Ciphertexts

Input: $\overrightarrow{CN_i}, \overrightarrow{NN_{j,i}}$ for $j \in \mathcal{U}_1, j \neq i$, $Cek_i = (B, M_i)$, $\{Nek_j\}_{j \in \{1, \dots, |\mathcal{U}_1|, j \neq i\}}$ where $Nek_j = (B^*, Q_j)$.

Output: ciphertexts $\{CenC_{i,j}\}_{j \neq i, j \in \mathcal{U}_1}$ and $\{NeiD_{j,i}\}_{j \neq i, j \in \mathcal{U}_1}$.

- 1: each $\mathcal{DS}_i (i \in \mathcal{U}_1)$ do:
- 2: $\vec{u} \leftarrow \overrightarrow{CN_i} = (1, x_c, y_c, x_c^2 + y_c^2, -1); (\mu_\rho)_\rho \leftarrow M_i$.
- 3: **for** index $j \leftarrow 1$ to $|\mathcal{U}_1|$ and $j \neq i$ **do**
- 4: Sample $\vec{\sigma} \xleftarrow{\$} (\mathbb{Z}_p^*)^3$ and set $C_0 \leftarrow \vec{\sigma} \cdot G_1$.
- 5: **for** each element in vector $\rho = 1, \dots, 5$ **do**
- 6: $\lambda_\rho = Hash(l_{i,j}, \rho) \in \mathbb{Z}_p; \pi_\rho \xleftarrow{\$} \mathbb{Z}_p$.
- 7: $C_\rho \leftarrow (\pi_\rho(\vec{b}_1 + \lambda_\rho \cdot \vec{b}_2) + u_\rho \vec{b}_3 + \sum_{t=1}^3 \sigma_t \mu_{\rho,t} \cdot \overrightarrow{b_{4+2t}})$.
 $G_1 = (\pi_\rho, \pi_\rho \lambda_\rho, u_\rho, 0, 0, \sigma_1 \mu_{\rho,1}, \sigma_2 \mu_{\rho,2}, \sigma_3 \mu_{\rho,3}, 0, 0)_{\mathbb{B}}$.
- 8: **end for**
- 9: Generate ciphertext $CenC_{i,j} \leftarrow (l_{i,j}, C_0, (C_\rho)_\rho)$.
- 10: $\vec{v} \leftarrow \overrightarrow{NN_{j,i}} = (x_n^2 + y_n^2, -2x_n, -2y_n, 1, r_{j,i}); (\delta_\rho)_\rho \leftarrow Q_j$.
- 11: Sample $\vec{\xi} \xleftarrow{\$} (\mathbb{Z}_p^*)^3$ and set $D_0 \leftarrow \vec{\xi} \cdot G_2$.
- 12: **for** each element in vector $\rho = 1, \dots, 5$ **do**
- 13: $\lambda_\rho = Hash(l_{j,i}, \rho) \in \mathbb{Z}_p; \gamma_\rho \xleftarrow{\$} \mathbb{Z}_p$.
- 14: $D_\rho \leftarrow (\gamma_\rho(\lambda_\rho \cdot \vec{b}_1^* - \vec{b}_2^*) + v_\rho \vec{b}_3^* + \sum_{t=1}^3 \xi_t \delta_{\rho,t} \cdot \overrightarrow{b_{4+2t}^*})$.
 $G_2 = (\gamma_\rho \lambda_\rho, -\gamma_\rho, v_\rho, 0, 0, \xi_1 \delta_{\rho,1}, \xi_2 \delta_{\rho,2}, \xi_3 \delta_{\rho,3}, 0, 0)_{\mathbb{B}^*}$.
- 15: **end for**
- 16: Generate ciphertext $NeiD_{j,i} \leftarrow (l_{j,i}, D_0, (D_\rho)_\rho)$.
- 17: **end for**
- 18: Send ciphertexts $\{CenC_{i,j}\}_{j \neq i, j \in \mathcal{U}_1}$ and $\{NeiD_{j,i}\}_{j \neq i, j \in \mathcal{U}_1}$ to $\mathcal{S}_1$ in secure communication channel.

Protocol 3 Decryption Phases

Input: $CenC_{i,j}, NeiD_{j,i}$; decryption key dk_i .

Output: the inner product $s_{i,j}$.

- 1: $\mathcal{S}_1$ receives the ciphertexts.
- 2: **for** $i \in \mathcal{U}_1, j \in \mathcal{U}_1$ and $j \neq i$ **do**
- 3: $\vec{d} \leftarrow dk_i = \sum_{\rho=1}^5 (\mu_{\rho,1} \delta_{\rho,1}, \mu_{\rho,2} \delta_{\rho,2}, \mu_{\rho,3} \delta_{\rho,3}) \in \mathbb{Z}_p^3$
- 4: $(l, C_0, (C_\rho)_\rho) \leftarrow CenC_{i,j}; (l', D_0, (D_\rho)_\rho) \leftarrow NeiD_{j,i}$
- 5: **for** each element in vector $\rho = 1, \dots, 5$ **do**
- 6: **if** labels $l = l'$ **then**
- 7: Multiply ciphertexts $C_\rho \times D_\rho = g_t^{u_\rho v_\rho} \cdot \prod_q (g_t^{\mu_{\rho,q} \delta_{\rho,q}})^{\xi_q \sigma_q}$
- 8: **end if**
- 9: **end for**
- 10: Continuously multiply $C_\rho \times D_\rho$ to obtain $\prod_\rho C_\rho \times D_\rho = g_t^{\sum_\rho u_\rho v_\rho} \cdot \prod_q e(C_{0,q}, D_{0,q})^{d_q}$
- 11: Compute the inner product $\sum_\rho u_\rho v_\rho = \log_{g_t} \left(\frac{\prod_\rho C_\rho \times D_\rho}{\prod_q e(C_{0,q}, D_{0,q})^{d_q}} \right)$, denoted by $s_{i,j}$.
- 12: **end for**

$g_t = e(G_1, G_2)$, $\mathcal{S}_1$ continuously multiplies $C_\rho \times D_\rho$ to get $\prod_\rho C_\rho \times D_\rho = g_t^{\sum_\rho u_\rho v_\rho} \cdot \prod_q e(C_{0,q}, D_{0,q})^{d_q}$. It is obvious that $\sum_\rho u_\rho v_\rho = \overrightarrow{CN_i} \circ \overrightarrow{NN_{j,i}} = s_{i,j}$. Thus, $\mathcal{S}_1$ could compute the inner product of vector pairs, as

$$s_{i,j} = (\log g_t) \left(\prod_\rho C_\rho \times D_\rho / \prod_q e(C_{0,q}, D_{0,q})^{d_q} \right),$$

Protocol 4 Oblivious Data Collecting

Input: $r_{i,j}, s_{i,j}, X_i$.

Output: $[X]_1, [X]_2$.

- 1: **for** $\mathcal{DS}_j (j \in \mathcal{U}_1)$; index $i \in \mathcal{U}_1$ and $j \neq i$ **do**
- 2: $\mathcal{DS}_j$ calculates $z_{i,j} \leftarrow (ffs_{i,j} \cdot c \cdot \Delta t)^2 - r_{i,j}$ locally.
- 3: $\mathcal{DS}_j$ and $\mathcal{S}_1$ perform $\pi_{Compare}$ on $z_{i,j}, s_{i,j} \in \mathbb{Z}_2$. $\mathcal{DS}_j$ sends output $[FFR_{i,j}]_1$ to $\mathcal{S}_1$ who rebuilds the result $FFR_{i,j}$.
- 4: **end for**
- 5: $\mathcal{S}_1$ calculates $\sum_{j=1}^{|\mathcal{U}_1|} FFR_{i,j}$ and get $CR_i \in \{0, 1\}$ for each $\mathcal{DS}_i$.
- 6: $\mathcal{S}_1$ sends the bits $\{CR_i\}_{i \in \mathcal{U}}$ to $\mathcal{S}_2$.
- 7: **for** $\mathcal{DS}_i (i \in \mathcal{U})$ **do**
- 8: $\mathcal{DS}_i$ additively shares X_i into $\{[X_i]_1, [X_i]_2\}$.
- 9: $\mathcal{DS}_i$ invoke OT with $\mathcal{S}_1$ and $\mathcal{S}_2$ respectively. $\mathcal{DS}_i$ is a sender with inputs $\{m_{i,0}^1, m_{i,1}^1\} \leftarrow \{\perp, [X_i]_1\}$; $\{m_{i,0}^2, m_{i,1}^2\} \leftarrow \{\perp, [X_i]_2\} \parallel \{r_{j,i}\}_{j \in \mathcal{U}}$. $\mathcal{S}_1$ and $\mathcal{S}_2$ are receivers with CR_i .
- 10: **end for**
- 11: $\mathcal{S}_1$ and $\mathcal{S}_2$ remove $\mathcal{DS}$ s who send $\perp$, and forms a new set $\mathcal{U}_2$. If $i \notin \mathcal{U}_2$, set $X_i = (0, \dots, 0)$.
- 12: $\mathcal{S}_b (b \in \{1, 2\})$ forms the shared feature matrix $[X]_b \in R_2^{|\mathcal{U}| \times P}$, where $X = [X]_1 + [X]_2$, $X = [X_1, X_2, \dots, X_{|\mathcal{U}_1|}]$.

which is also the masked squared distance as $s_{i,j} = dist_{i,j}^2 - r_{i,j}$. Here, $r_{i,j}$ is a random mask held by $\mathcal{DS}_j$, so $(s_{i,j}, r_{i,j})$ form additive secret shares of the true squared distance $dist_{i,j}^2$.

⊙ Oblivious data collecting

The oblivious data collecting strategy is defined in line with Free-Flow Reachability [21], which characterizes the capability of traveling at free-flow speed (i.e., without congestion) to another location within a specified time threshold. This formulation facilitates the identification of isolated $\mathcal{DS}$ s and constrains feature collection to nodes that lie within their reachable influence domain. Oblivious data collecting strategy. Assume that $ffs_{i,j}$ is the free-flow speed between $\mathcal{DS}_i$ and $\mathcal{DS}_j$, meaning the average speed in the absence of congestion or other adverse conditions. Δt is the duration of time quantum, c is a number of time intervals considered. $FFR_{i,j}$ is the free-flow reachability from $\mathcal{DS}_i$ to $\mathcal{DS}_j$. The collecting strategy CR_i for $\mathcal{DS}_i (i \in \mathcal{U}_1)$ is

$$CR_i = \begin{cases} 1, & \sum_{j=1}^{|\mathcal{U}_1|} FFR_{i,j} > 0; \\ 0, & \text{otherwise,} \end{cases} \quad (4)$$

where $FFR_{i,j} = 1$ if $(ffs_{i,j} \cdot c \cdot \Delta t)^2 - dist_{i,j}^2 \geq 0$, else 0.

Consequently, a $\mathcal{DS}$ with $CR_i = 0$ is isolated, meaning that its location is not reachable from any other $\mathcal{DS}$. For many graph-based tasks, such isolated nodes contribute little value while increasing storage and communication costs. Moreover, collecting features from these nodes expands the potential privacy risk. In line with the Data Minimization Principle in GDPR [22] which requires collecting the minimum amount of personal data to achieve the defined purpose, features are collected solely from nodes with $CR = 1$.

The detailed process is presented in Protocol 4, and its visualization is illustrated in Fig. 6. Firstly, to compute $FFR_{i,j}$,

a comparison between $(ff_{s_{i,j}} \cdot c \cdot \Delta t)^2 - dist_{i,j}^2$ and 0 is needed. Then, this comparison is converted into that between $z_{i,j}$ and $s_{i,j}$, where $z_{i,j} = (ff_{s_{i,j}} \cdot c \cdot \Delta t)^2 - r_{i,j}$. To avoid leaking any information about distances, $\mathcal{S}_1$ and $\mathcal{DS}_j$ execute a two-party secure protocol $\pi_{Compare}$ [13] listed in TABLE I. With the inputs $z_{i,j}$ from $\mathcal{DS}_j$ and $s_{i,j}$ from $\mathcal{S}_1$, $\pi_{Compare}$ output a bit $FFR_{i,j}$ following $FFR_{i,j} = 1$ if $z_{i,j} \geq s_{i,j}$, 0 otherwise. Following this, $\mathcal{S}_1$ sums to generate the collecting strategy list $\{CR_i\}_{i \in \mathcal{U}}$ and shares to $\mathcal{S}_2$. Next, servers collect features from $\mathcal{DS}$ s obviously using the collecting strategies.

The feature vector held by $\mathcal{DS}_i$ is expressed as $X_i \in R^{1 \times P}$, where P represents the number of node attribute features. $\mathcal{DS}_i$ splits X_i into $[X_i]_1, [X_i]_2$ based on additive secret sharing, which ensures feature privacy.

Then, oblivious transfer (OT) is used to collect shared feature vectors. $\mathcal{DS}$ acts as a sender while $\mathcal{S}$ acts as a receiver with bit CR . In this way, only attribute features from reachable $\mathcal{DS}$ s are collected by servers. Also, the privacy of CR ensures $\mathcal{DS}$ s are unable to know if their shared features have been collected, thus protecting the privacy of reachability as well as distance correlations between nodes. As Line 9, the difference between two inputs is that masks $\{r_{j,i}\}_{j \in \mathcal{U}_1}$ together with shared features are sent to $\mathcal{S}_2$. In this way, $\mathcal{S}_2$ holds masks r , while $\mathcal{S}_1$ holds masked distances s . Finally, servers form the shared feature matrix $[X]$ separately.

③ Privacy preserving adjacency matrix generation

The adjacency matrix A , which encodes edge weights between nodes, serves as a key mechanism for capturing spatial dependencies [23]. To characterize the proximity between two nodes and model the decay of spatial dependency, the threshold Gaussian kernel is used to construct the adjacency matrix [24]. σ is the standard deviation of all distances and κ is a threshold to ensure sparsity of A . For $i, j \in \mathcal{U}_1$:

$$A_{i,j} = \begin{cases} \exp\left(-\frac{dist_{i,j}^2}{\sigma^2}\right), & dist_{i,j}^2 \leq \kappa; \\ 0, & otherwise. \end{cases} \quad (5)$$

The first challenge is to compute the variance of all distances σ^2 from shares of the squared distances held by servers. According to probability theory and statistics, σ^2 can be obtained from the expected values as $\sigma^2 = E(dist^2) - E(dist)^2$.

In Protocol 5, line 4 computes the mathematical expectation of squared distances $E(dist^2)$ easily, because it is the mean of the sum. $E(dist^2)$ contains no sensitive information about any specific $\mathcal{DS}$, thus can be revealed to servers. While computing $E(dist)$ needs to compute the average of all $dist$ s. Inputting squared distance shares, the secure square root protocol π_{Sqrt} [14] helps servers to compute $dist$, avoiding the leakage of distances. Specifically, with inputting shares $s_{i,j}$ and $r_{i,j}$ where $s_{i,j} + r_{i,j} = dist_{i,j}^2$, servers perform π_{Sqrt} and output shares of $dist_{i,j}$. Similar to $E(dist^2)$, they sum, exchange, add, and take the average to get $E(dist)$. This process does not compromise individual distance privacy.

With σ^2 , for each pair (i, j) , servers execute $\pi_{compare}$ with input $s_{i,j}$ and $\kappa - r_{i,j}$ to judge $dist_{i,j}^2 \leq \kappa$ without leaking the distances. Then, they locally and separately construct CAI and CAZ which are the element-wise multiplication factors of the

Protocol 5 Privacy Preserving Adjacency Matrix Generation

Input: $r_{i,j}, s_{i,j} \in \mathbb{Z}_{2^l}$.

Output: Element-wise multiplication factors CAI, CAZ .

- 1: $\mathcal{S}_1$ sum shares as $Sumsr_1 \leftarrow \sum_{i,j \in \mathcal{U}_2} s_{i,j}$.
- 2: Similarly, $\mathcal{S}_2$ gets $Sumsr_2 \leftarrow \sum_{i,j \in \mathcal{U}_2} r_{i,j}$.
- 3: $\mathcal{S}_1$ and $\mathcal{S}_2$ add up to rebuild $Sumsr = \sum_{i,j \in \mathcal{U}_2} dist^2$.
- 4: Each server sets $E(dist^2) \leftarrow Sumsr/|\mathcal{U}_2|$.
- 5: $\mathcal{S}_1$ and $\mathcal{S}_2$ execute π_{Sqrt} with inputs $\{s_{i,j}, r_{i,j}\}_{i,j \in \mathcal{U}_1, i \neq j}$ and achieve the outputs $\{opt1_{i,j}, opt2_{i,j}\}_{i,j \in \mathcal{U}_1, i \neq j}$.
- 6: $\mathcal{S}_1$ and $\mathcal{S}_2$ separately sum up the outputs locally, then exchange to rebuild the secret, denoted as $SumsqrtOut$.
- 7: Each server sets $E(dist) \leftarrow SumsqrtOut/|\mathcal{U}_2|$.
- 8: $\mathcal{S}_1$ and $\mathcal{S}_2$ can easily compute the variance σ^2 which complies with $\sigma^2 = E(dist^2) - E(dist)^2$.
- 9: $\mathcal{S}_1$ and $\mathcal{S}_1$ each construct an $|\mathcal{U}_1| \times |\mathcal{U}_1|$ matrix of all zeros, named CAI, CAZ respectively.
- 10: **for** each pair $i \in \mathcal{U}_1, j \in \mathcal{U}_1, j \neq i$ **do**
- 11: Execute $\pi_{compare}$ on the $s_{i,j}$ and $\kappa - r_{i,j}$. If $s_{i,j} \leq \kappa - r_{i,j}$, $\mathcal{S}_1$ sets $CAI_{i,j} = \exp(-\frac{s_{i,j}}{\sigma^2})$; $\mathcal{S}_2$ sets $CAZ_{i,j} = \exp(-\frac{r_{i,j}}{\sigma^2})$.
- 12: **end for**
- 13: $\mathcal{S}_1$ holds CAI while $\mathcal{S}_2$ holds CAZ , where $A = CAI \otimes CAZ$.

adjacency matrix, where $A = CAI \otimes CAZ$. Thus, the adjacency matrix $A \in \mathbb{R}_{2^l}^{|\mathcal{U}_1| \times |\mathcal{U}_1|}$ is generated. Furthermore, it can be converted into the form of additive secret sharing of A , as introduced in secure aggregation module.

Considering the dynamics of mobile nodes (data suppliers) which may involve joining, leaving, or location shift, specific update mechanisms are required for the secret-shared adjacency matrix $[A]$ and feature matrix $[X]$. When a node leaves, the servers only need to delete the elements related to it from the $[A]$ and $[X]$. When a node $\mathcal{DS}_i$ joins, the server and data suppliers perform the private distance computation between the newly joined node and other nodes, then the servers collect shared new features $[X_i]$, compute the corresponding elements $CAI_{i,j}, CAZ_{i,j} (j \in \mathcal{U}_1)$ in the adjacency matrix, and update these elements into $[X]$ and $[A]$ to facilitate subsequent operations. Furthermore, considering that potential location shift of nodes may interfere with the spatial attributes of features, the hash of a timestamp can be submitted along with the ciphertext of location or feature. Consistency between features and locations is ensured by comparing the hash values.

④ Secure aggregation

1) *A simplified and universal aggregation:* $\mathcal{S}_1$ and $\mathcal{S}_2$ with $\mathcal{S}_0$ who has no inputs or outputs use MPC protocols in TABLE I to transform general aggregation functions to secure ones. Based on π_{MatMul} and $\pi_{EWMatMul}$, the aggregation function Equation (2) is expressed as follows.

$$\begin{aligned} AX &= (CAI \otimes CAZ) X \\ &= ([CAI]_1 + [CAI]_2) \otimes ([CAZ]_1 + [CAZ]_2) X \\ &= ([A]_1 + [A]_2) ([X]_1 + [X]_2) \end{aligned} \quad (6)$$

$\mathcal{S}_1$ and $\mathcal{S}_2$ additively share their private element-wise multiplication factors of A, CAI and CAZ , to each other. Then, inputting such shares, $\pi_{EWMatMul}$ outputs the shared

Protocol 6 Secure Aggregation in GCN**Input:** $CAI, CAz, [X]_b, b \in \{1, 2\}$.**Output:** $[H]_b, b \in \{1, 2\}$.

- 1: S_1, S_2 add I_N to CAI and CAz separately, generate $\widetilde{CAI}$ and $\widetilde{CAz}$.
- 2: S_1, S_2 additive secret share $\widetilde{CAI}$ and $\widetilde{CAz}$ to each other.
- 3: $[A]_b \leftarrow \pi_{EWMatMul}(\widetilde{CAI})_b, [\widetilde{CAz}]_b$ with $S_0, b \in \{1, 2\}$.
- 4: S_b computes $[\widetilde{D}]_b = \sum_j [A_{i,j}]_b, b \in \{1, 2\}$.
- 5: S_1 and S_2 exchange the shares and reconstruct $\widetilde{D}$.
- 6: S_b compute $[\widehat{A}]_b = \widetilde{D}^{-\frac{1}{2}} [A]_b \widetilde{D}^{-\frac{1}{2}}, b \in \{1, 2\}$.
- 7: $[H]_b \leftarrow \pi_{MatMul}([\widehat{A}]_b, [X]_b), b \in \{1, 2\}$, where $H = \widehat{A}X$.

adjacency matrix $[A]_1, [A]_2$, where $A = [A]_1 + [A]_2$. Next, $\pi_{MatMul}([A]_1, [X]_1, [A]_2, [X]_2)$ outputs the shares of AX .

2) *An improved aggregation:* Similar to the simplified aggregation, S_1 and S_2 use π_{MatMul} and $\pi_{EWMatMul}$ to securely compute Equation (3). Protocol 6 introduces more details.

$$\begin{aligned}
 \widehat{A}X &= \widetilde{D}^{-\frac{1}{2}} \widetilde{A} \widetilde{D}^{-\frac{1}{2}} X \\
 &= \left(\sum_{b=1}^2 \left(\sum_{p=1}^2 [\widetilde{D}]_p \right)^{-\frac{1}{2}} [\widehat{A}]_b \left(\sum_{p=1}^2 [\widetilde{D}]_p \right)^{-\frac{1}{2}} \right) X \\
 &= ([\widehat{A}]_1 + [\widehat{A}]_2) ([X]_1 + [X]_2)
 \end{aligned} \quad (7)$$

Specifically, with $A = CAI \otimes CAz$, and given that the diagonal elements of CAI and CAz are zero, it follows that $\widehat{A} = A + I_N = (CAI + I_N) \otimes (CAz + I_N)$. To ensure adjacency matrix privacy, servers utilize additive secret sharing and $\pi_{EWMatMul}$ to output the shared $\widehat{A}$. Then, S_b can compute $[\widetilde{D}]_b$ respectively as Line 4. As shown in Equation (7), they reconstruct $\widetilde{D}$ and compute $[\widehat{A}]_b$ locally. Finally, with π_{MatMul} , servers output the shares of AX securely.

C. Collusion-Resilient Enhancement

To enhance the robustness of our protocol against collusion between the two servers, we introduce an additional multiplicative blinding mechanism based on random masks.

In setup phase, all $\mathcal{DS}$ s agree on a session key K_s , which can either be generated by a trusted key distribution center (KDC) or established via a distributed key agreement protocol. For large-scale deployments, the latter may incur substantial computational overhead. Using K_s as the seed and incorporating task-specific labels or salts (e.g., one for positional vector blinding, another for feature vector blinding), a pseudorandom function (PRF) derives two independent positive blinding factors α and β . These masks are computationally indistinguishable from truly random values without knowledge of the seed, thereby enhancing both security and key reusability.

In addition to the operations in Protocol 1, each $\mathcal{DS}$ applies multiplicative blinding to its vector $\overrightarrow{NN}$ with mask α :

$$\overrightarrow{NN'} = (\alpha x_n^2 + \alpha y_n^2, -2\alpha x_n, -2\alpha y_n, \alpha, \alpha r).$$

Then, with IPFE, S_1 decrypts encrypted vectors to obtain the doubly-masked squared distance $s' = \alpha \cdot \text{dist}^2 - \alpha r$.

In the oblivious data collecting, rather than computing z as in Protocol 4, each $\mathcal{DS}$ locally computes: $z' \leftarrow \alpha(f f s \cdot c \cdot$

$\Delta t)^2 - \alpha r$ locally. Since $\alpha > 0$, the comparison between z' and $r' = \alpha r$ is equivalent to that between z and r . Thus, S_1 outputs the collection result CR and shares to S_2 . Before performing OT with servers, each $\mathcal{DS}$ blinds its feature vector X_i with mask β . Even if the servers collude and exchange data, they only obtain βX , which does not reveal the true features.

When generating adjacency matrix, S_1 and S_2 respectively hold the masked distances shares $\{r'\}$ and $\{s'\}$. If they collude to sum the shares, they can obtain:

$$s' + r' = \alpha \cdot \text{dist}^2 - \alpha r + \alpha r = \alpha \cdot \text{dist}^2,$$

where α remains unknown for servers. Therefore, the multiplicative blinding with α ensures that the true distances remain hidden. Then, we observe that both $E(\alpha \cdot \text{dist}^2)$ and $E(\alpha \cdot \text{dist})$ can be readily computed according to Protocol 5. The value σ^2 can be obtained in masked form as: $\sigma'^2 = E(\alpha \cdot \text{dist}^2) - E(\alpha^{1/2} \cdot \text{dist})^2 = \alpha \sigma^2$. Notably, the following relations hold:

$$\frac{s'_{i,j}}{\sigma'^2} = \frac{s_{i,j}}{\sigma^2}, \frac{r'_{i,j}}{\sigma'^2} = \frac{r_{i,j}}{\sigma^2},$$

which ensures the correctness of the adjacency computation despite the blinding factor. Specifically, S_1 sets $CAI_{i,j} = \exp(-\frac{s'_{i,j}}{\sigma'^2})$, S_2 sets $CAz_{i,j} = \exp(-\frac{r'_{i,j}}{\sigma'^2})$. However, if CAI and CAz are exchanged between colluding servers, they can reconstruct the adjacency matrix, meaning that adjacency privacy is not preserved under servers collusion model.

Collusive servers perform aggregation on feature matrix and adjacency matrix, the aggregated result is βAX . The $\mathcal{DS}$ s must reveal β , then one could obtain the aggregated result.

Security Properties. Location privacy and collecting strategy privacy rely solely on IPFE and OT, respectively; their guarantees are identical in the colluding and non-colluding settings. Distance and feature privacy are enforced by multiplicative masks (α, β); collusion only aggregates transcripts but does not help invert unknown PRF-derived scalars. Nevertheless, the main limitation is the loss of adjacency matrix privacy in the server collusion model, as the masking does not prevent reconstruction from CAI and CAz . Addressing this remains an open direction for future work.

V. SECURITY ANALYSIS**A. Location Privacy**

Let $\mathcal{A}$ denote an adversary who aims to acquire locations. $\mathcal{A}$ might be S_1 or an external attacker. If location privacy can be achieved while $\mathcal{A}$ is S_1 , it can also be assured while the adversary is external. So, we focus on the situation when $\mathcal{A}$ is S_1 instead of discussing the external adversary individually. For the sake of clarity, we limit the number of $\mathcal{DS}$ s to 2.

Definition 1 (Location privacy) A scheme Π is location privacy if for every PPT adversary $\mathcal{A}$ there is a negligible function $\text{negl}(\cdot)$ such that for a sufficiently large λ ,

$$\text{Prob}[(\overrightarrow{CN}, \overrightarrow{NN}) \leftarrow A_\Pi(\text{CenC}, \text{NeiD}, dk, s) :$$

$$s = \overrightarrow{CN} \circ \overrightarrow{NN}] \leq \text{negl}(\lambda).$$

Theorem 1: Assuming IPFE satisfies message privacy, PEDAs is location privacy.

Proof. When $\mathcal{A}$ could output $\overrightarrow{CN}, \overrightarrow{NN}$, $\mathcal{A}$ gains locations $(x_c, y_c), (x_n, y_n)$. Given the ciphertexts $CenC, NeiD$, a decryption key dk , we assume $\mathcal{A}$ could output $\overrightarrow{CN}, \overrightarrow{NN}$ with a non-negligible probability ε_1 . Then, we construct an adversary $\mathcal{B}$ from a game of IPFE. The $\mathcal{B}$ sets $CenC = X, NeiD = Y, dk = k, z = s$ and runs a copy of PEDDA. When $\mathcal{A}$ outputs $\overrightarrow{CN}, \overrightarrow{NN}$, do: if $\overrightarrow{CN} = \vec{x}, \overrightarrow{NN} = \vec{y}$, $\mathcal{B}$ outputs $\vec{x}, \vec{y}$. Else, Sample locations $(x_c, y_c, x_n, y_n) \xleftarrow{R} \mathbb{Z}_p$ and form $\vec{x}$ and $\vec{y}$. In this way, we have the probability of $\mathcal{B}$ outputting $\vec{x}, \vec{y}$ with ciphertexts and decryption key is $\varepsilon_{IPFE} = \varepsilon_1 + (1 - \varepsilon_1) \cdot \frac{1}{p^4}$. Thus, ε_{IPFE} is non-negligible, which contradicts IPFE - Message privacy. q is sufficiently large, we have:

$$\begin{aligned} \varepsilon_1 &= \text{Prob}[(\overrightarrow{CN}, \overrightarrow{NN}) \leftarrow \mathcal{A}_{\Pi}(CenC, NeiD, dk, s)] \\ &\approx \varepsilon_{IPFE} - \frac{1}{p^4} \leq \text{negl}(\lambda), \end{aligned}$$

which proves the location privacy of our scheme. $\square$

B. Distance Privacy

We assume that $\mathcal{A}$ is an adversary who aims to capture the distance $dist$. $\mathcal{A}$ may be $\mathcal{S}_1, \mathcal{S}_2, \mathcal{DS}$, or an external attacker. We assume that $\mathcal{A}$ is $\mathcal{S}_1$. For the sake of clarity, we assume that there is only one distance between the two locations.

Definition 2 (Distance privacy) A scheme Π is distance privacy if for every PPT adversary $\mathcal{A}$ there is a negligible function $\text{negl}(\cdot)$ such that for a sufficiently large λ ,

$$\begin{aligned} \text{Prob}[dist \leftarrow \mathcal{A}_{\Pi}(CenC, NeiD, dk, s, opt1) : dist^2 \\ = s + r = \overrightarrow{CN} \circ \overrightarrow{NN} + r] \leq \text{negl}(\lambda), \end{aligned}$$

Theorem 2: Assuming $\pi_{Compare}$ and $\pi_{S_{qrt}}$ [13], [14] are MPC protocols in a semi-honest model, PEDDA is distance privacy.

Proof. When $\mathcal{A}$ outputs $dist$, given three possible cases.

Case 1: $\mathcal{A}$ could gain locations and compute their distance.

Case 2: $\mathcal{A}$ could figure out r , compute $dist^2 = s + r$.

Case 3: $\mathcal{A}$ could obtain $opt2$, rebuild $dist = opt1 + opt2$.

In the following part, we prove that the probability of the occurrence of any one of the above three cases is negligible.

Case 1. Assuming $\mathcal{A}$ could obtain two locations, it is obvious that the distance can be calculated. However, we have already proven location privacy, so this assumption conflicts with our proof. So, the probability is $\varepsilon_{21} \approx \varepsilon_1 - \frac{1}{p^4}$.

Case 2. Let ε_{22} denote the probability that $\mathcal{A}$ seeks r . We analyze four probabilities as $\varepsilon_{22} = \varepsilon_{221} + \varepsilon_{222} + \varepsilon_{223} + \varepsilon_{224}$. (1) If $\mathcal{A}$ obtain $\overrightarrow{NN}$, then $\mathcal{A}$ can extract r , due to $\overrightarrow{NN} = (x_n^2 + y_n^2, -2x_n, -2y_n, 1, r)$. While we have proven that such probability is negligible in Theorem 6. Thus, $\varepsilon_{221} \approx \varepsilon_1 - \frac{1}{p}$. (2) $\mathcal{A}$ receives r from $\mathcal{DS}$ and $\mathcal{S}_2$. However, this contradicts our assumption. Both $\mathcal{DS}$ and $\mathcal{S}_2$ are honest but curious and non-collusive. Therefore, they do not send the element r to $\mathcal{A}$. Thus, $\varepsilon_{222} = \frac{1}{p}$. (3) $\mathcal{A}$ captures r from an MPC protocol $\pi_{Compare}$ [13]. In Protocol 5, $\pi_{Compare}$ inputs $z = (f \cdot s \cdot c \cdot \Delta t)^2 - r$ from $\mathcal{DS}$ and s from $\mathcal{A}$. If $\mathcal{A}$ could obtain $\mathcal{DS}$'s input z , $\mathcal{A}$ could compute r . Assuming such probability is a non-negligible ε_{223} . We have a non-negligible $\varepsilon_{MPC} = \varepsilon_{223} + (1 - \varepsilon_{223}) \cdot \frac{1}{p}$, which contradicts MPC. Therefore, $\varepsilon_{223} \approx \varepsilon_{MPC} - \frac{1}{p}$. When computing the adjacency matrix,

$\pi_{Compare}$ is executed on the inputs $s_{i,j}$ and $\kappa - r_{i,j}$. Same as above, without further elaboration. (4) $\mathcal{A}$ captures r from an MPC protocol $\pi_{S_{qrt}}$ [14]. $\pi_{S_{qrt}}$ inputs s from $\mathcal{A}$ and r from $\mathcal{S}_2$. If $\mathcal{A}$ could obtain $\mathcal{DS}$'s input, $\mathcal{A}$ could get r directly. Similar to Case 2.3, $\varepsilon_{224} \approx \varepsilon_{MPC} - \frac{1}{p}$. In summary, $\varepsilon_{22} \approx \varepsilon_1 + 2\varepsilon_{MPC} - \frac{2}{p}$.

Case 3. Let $\varepsilon_{23} = \varepsilon_{231} + \varepsilon_{232}$ denote the probability that $\mathcal{A}$ obtain $opt2$. (1) $\mathcal{S}_2$ sends $opt2$ to $\mathcal{A}$. As discussed before, this contradicts the security assumption. $\varepsilon_{231} = \frac{1}{2}$. (2) $\mathcal{A}$ can get $opt2$ from an MPC protocol $\pi_{S_{qrt}}$, which outputs $opt1$ to $\mathcal{A}$ and $opt2$ to $\mathcal{S}_2$. Assuming such probability is a non-negligible ε_{232} , we have a non-negligible $\varepsilon_{MPC} = \varepsilon_{232} + (1 - \varepsilon_{232}) \cdot \frac{1}{2}$, which contradicts MPC-Input privacy. Therefore, $\varepsilon_{232} \approx \varepsilon_{MPC} - \frac{1}{2}$ is negligible. In summary, $\varepsilon_{23} \approx \varepsilon_{MPC}$.

We know that the value p is sufficiently large, and $\varepsilon_1, \varepsilon_{MPC}$ are negligible. Then, we have:

$$\begin{aligned} \text{Prob}[dist \leftarrow \mathcal{A}_{\Pi}(CenC, NeiD, dk, s, opt1)] &= \varepsilon_{21} \\ &+ \varepsilon_{22} + \varepsilon_{23} \approx 2\varepsilon_1 + 3\varepsilon_{MPC} - \frac{2}{p} - \frac{1}{p^2} \leq \text{negl}(\lambda), \end{aligned}$$

which proves the distance privacy of our scheme. $\square$

C. Adjacency Matrix Privacy

We assume that $\mathcal{A}$ is an adversary who aims to capture the adjacency matrix A . $\mathcal{A}$ might be $\mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_0$ or an external attacker. We focus on that $\mathcal{A}$ corrupts $\mathcal{S}_1$.

Definition 3 (Adjacency matrix privacy) A scheme Π is adjacency matrix privacy if for every PPT adversary $\mathcal{A}$ there is a negligible function $\text{negl}(\cdot)$ with a sufficiently large λ ,

$$\begin{aligned} \text{Prob}[A \leftarrow \mathcal{A}_{\Pi}(CAI, [CAz]_1, [A]_1, [\widetilde{CAz}]_1, [\widetilde{A}]_1, [\widehat{A}]_1) : \\ A = CAI \otimes CAz] \leq \text{negl}(\lambda). \end{aligned}$$

Theorem 3: Assuming additive secret sharing satisfies perfect privacy, π_{MatMul} and $\pi_{EWMatMul}$ are MPC protocols in a semi-honest model, our PEDDA is adjacency matrix privacy.

Proof. When $\mathcal{A}$ could output A , given there are three possible cases as follows. We simplify the proof for some cases analogous to those previously analyzed.

Case 1: $\mathcal{A}$ could get CAz , which lead to $A = CAI \otimes CAz$.

Case 2: $\mathcal{A}$ could get $[A]_2$, and compute $A = [A]_1 + [A]_2$.

Case 3: $\mathcal{A}$ could obtain variants of A .

Next, we prove that the probability of the occurrence of any one of them is negligible.

Case 1. Let $\varepsilon_{31} = \varepsilon_{311} + \varepsilon_{312} + \varepsilon_{313} + \varepsilon_{314}$ denote the probability getting CAz . (1) $\mathcal{A}$ has r and computes CAz as line 11 in Protocol 5. However, in Theorem 2, we have proven that ε_{22} , the probability of $\mathcal{A}$ obtaining r , is negligible. So, $\varepsilon_{311} \approx \varepsilon_{22} - 2^{-l|\mathcal{U}_1|^2}$. (2) $\mathcal{A}$ works out $[CAz]_2$ or CAz from $[CAz]_1$, where $[CAz]_1$ and $[CAz]_2$ are additive shares of CAz . Assuming ε_{312} is non-negligible, we could construct an adversary in a secret sharing with $\varepsilon_{SS} = \varepsilon_{312} + (1 - \varepsilon_{312}) \cdot 2^{-l|\mathcal{U}_1|^2}$, which is also non-negligible. This contradicts SS-Perfect privacy. Therefore, $\varepsilon_{312} \approx \varepsilon_{SS} - 2^{-l|\mathcal{U}_1|^2}$. (3) $\mathcal{A}$ draws $[CAz]_2$ from $\pi_{EWMatMul}$ [16], which is an MPC protocol satisfying the MPC-Input privacy. $[CAz]_2$ is an input of $\pi_{EWMatMul}$. Then, similar to the proof of Theorem 2, we have $\varepsilon_{313} \approx \varepsilon_{MPC} - 2^{-l|\mathcal{U}_1|^2}$. (4) $\mathcal{A}$ receives $[CAz]_2$ or CAz from $\mathcal{S}_2$. As discussed earlier, this

contradicts the security assumption. Thus, $\varepsilon_{314} \approx 2^{-l|\mathcal{U}_1|^2}$. In summary, $\varepsilon_{31} \approx \varepsilon_{22} + \varepsilon_{SS} + \varepsilon_{MPC} - 2^{-(l-1)|\mathcal{U}_1|^2}$.

Case 2. Let ε_{32} denote the probability that $\mathcal{A}$ get $[A]_2$. (1) $\mathcal{A}$ draws $[A]_2$ from $\pi_{EWMatMul}$. $[A]_2$ is other party's output, this contradicts MPC-Input privacy. We have $\varepsilon_{321} \approx \varepsilon_{MPC} - 2^{-l|\mathcal{U}_1|^2}$. (2) $\mathcal{A}$ draws $[A]_2$ from π_{MatMul} [15]. $[A]_2$ is one of π_{MatMul} 's input, this contradicts MPC-Input privacy. Like Case 1.3, we have $\varepsilon_{322} \approx \varepsilon_{MPC} - 2^{-l|\mathcal{U}_1|^2}$. (3) $\mathcal{A}$ receives $[A]_2$ from $\mathcal{S}_2$. This contradicts the security assumption. Thus, $\varepsilon_{323} \approx -2^{-l|\mathcal{U}_1|^2}$. In summary, $\varepsilon_{32} = \varepsilon_{321} + \varepsilon_{322} + \varepsilon_{323} \approx 2\varepsilon_{MPC} - 2^{-l|\mathcal{U}_1|^2}$.

Case 3. In 2-layer GCN, we use some variants of A . Let ε_{33} denote the probability that $\mathcal{A}$ get variants of A . (1) $\mathcal{A}$ works out $[\widetilde{CAz}]_2$ or $\widetilde{CAz}$ from an additive share $[\widetilde{CAz}]_1$, which contradicts SS-Perfect privacy. (2) $\mathcal{A}$ draws other party's input $[\widetilde{CAz}]_2$ from $\pi_{EWMatMul}$, which contradicts MPC-Input privacy. (3) $\mathcal{A}$ draws other party's output $[\widetilde{A}]_2$ from $\pi_{EWMatMul}$, which contradicts MPC-Input privacy. (4) $\mathcal{A}$ draws other party's input $[\widetilde{A}]_2$ from π_{MatMul} , which contradicts MPC-Input privacy. (5) $\mathcal{A}$ receives $[\widetilde{CAz}]_2$, $\widetilde{CAz}$, $[\widetilde{A}]_2$ or $[\widetilde{A}]_2$ from $\mathcal{S}_2$, which contradicts the security assumption. In summary, the probability is $\varepsilon_{33} \approx \varepsilon_{SS} + 3\varepsilon_{MPC} - 3 \cdot 2^{-l|\mathcal{U}_1|^2}$.

As a result, we know that the value l is sufficiently large, ε_{22} , ε_{SS} and ε_{MPC} are negligible. Then, we have:

$$\begin{aligned} \text{Prob}[A \leftarrow \mathcal{A}_\Pi(CA, [CAz]_1, [A]_1, [\widetilde{CAz}]_1, [\widetilde{A}]_1, [\widetilde{A}]_1)] \\ \approx \varepsilon_{22} + 2\varepsilon_{SS} + 6\varepsilon_{MPC} - 2^{-(l-1)|\mathcal{U}_1|^2} \leq \text{negl}(\lambda), \end{aligned}$$

which proves the adjacency matrix privacy of our scheme. $\square$

D. Collecting Strategy Privacy

We focus on the case that $\mathcal{A}$ corrupts a $\mathcal{DS}_j$ who aims to gain the collecting strategy CR_j .

Definition 4 (Collecting strategy privacy) A scheme Π is collecting strategy privacy if for every PPT adversary $\mathcal{A}$ there is a negligible function $\text{negl}(\cdot)$ with a sufficiently large λ ,

$$\begin{aligned} \text{Prob}[CR_j \leftarrow \mathcal{A}_\Pi(\{[FFR_{i,j}]_1\}_{i \in \mathcal{U}_1}, \{m_0^1, m_1^1\}) : \\ m_{CR}^1 \leftarrow \text{OT}(m_0^1, m_1^1)] \leq \text{negl}(\lambda). \end{aligned}$$

Theorem 4: Assuming OT satisfies the receiver's security [12], our PEDAs is collecting strategy privacy.

Proof. When $\mathcal{A}$ could output CR_j , the possible cases are as follows.

Case 1. $\mathcal{A}$ could get $\{[FFR_{j,i}]_1\}_{i \in \mathcal{U}_1}, \{[FFR_{j,i}]_2\}_{i \in \mathcal{U}_1}$ from other $\mathcal{DS}$ s and $\mathcal{S}_1$, which contradicts security assumption. Therefore, $\varepsilon_{41} = 2^{-l(|\mathcal{U}_1|^2-1)}$.

Case 2. $\mathcal{A}$ could obtain CR_j . (1) $\mathcal{A}$ receive CR_j from $\mathcal{S}_1$ or $\mathcal{S}_2$. Similar to Case 1, $\varepsilon_{42} = \frac{1}{2}$. (2) $\mathcal{A}$ obtain CR_j during OT. $\mathcal{A}$ is the sender, while $\mathcal{S}_1$ is the receiver with CR_j . Assuming $\mathcal{A}$ has a non-negligible ε_{43} , then $\varepsilon_{OT} = \varepsilon_{43} + (1 - \varepsilon_{43}) \cdot \frac{1}{2}$ is clearly non-negligible, which contradicts OT-Receiver security. So, we have $\varepsilon_{43} = \varepsilon_{OT} - \frac{1}{2}$. The OT between $\mathcal{A}$ and $\mathcal{S}_2$ is similar, so we have $\varepsilon_{44} = \varepsilon_{OT} - \frac{1}{2}$.

As a result, we know that the value l is sufficiently large and $\varepsilon_{OT} - \frac{1}{2}$ is negligible. Then, we have:

$$\begin{aligned} \text{Adv}[CR_j \leftarrow \mathcal{A}_\Pi(\{[FFR_{i,j}]_1\}_{i \in \mathcal{U}_1}, \{m_0^1, m_1^1\})] = \varepsilon_{41} + \varepsilon_{42} \\ + \varepsilon_{43} + \varepsilon_{44} - \frac{1}{2} \approx 2\varepsilon_{OT} - 1 + 2^{-l(|\mathcal{U}_1|^2-1)} \leq \text{negl}(\lambda). \end{aligned}$$

In conclusion, $\mathcal{A}$ could only guess CR_j from $\{0, 1\}$, and there is no advantage beyond that. $\square$

E. Feature Privacy

Let $\mathcal{A}$ denote an adversary who aims to capture the features X_i or X . X is a feature matrix composed of multiple sets of vectors X_i , and we ensure the privacy of both X and X_i . The adversary $\mathcal{A}$ might be $\mathcal{S}_1$, $\mathcal{S}_2$, $\mathcal{S}_0$ or an external attacker. We focus on the circumstance in which the adversary $\mathcal{A}$ is $\mathcal{S}_1$.

Definition 5 (Feature privacy) A scheme Π is feature privacy if for every PPT adversary $\mathcal{A}$ there is a negligible function $\text{negl}(\cdot)$ such that for a sufficiently large λ ,

$$\begin{aligned} \text{Prob}[X_i, X \leftarrow \mathcal{A}_\Pi([X]_1, [X]_1) : X_i = [X]_1 \\ + [X]_2; X = [X]_1 + [X]_2] \leq \text{negl}(\lambda). \end{aligned}$$

Theorem 5: Assuming additive secret sharing satisfies perfect privacy, π_{MatMul} [15] is an MPC protocol in a semi-honest model, our PEDAs is feature privacy.

Proof. There are two possible cases. For cases similar to those previously analyzed, we simplify the proof.

Case 1. $\mathcal{A}$ could gain $[X]_2$. (1) $\mathcal{A}$ receives $[X]_2$ from $\mathcal{DS}_i$ or $\mathcal{S}_2$, which contradicts the security assumption. This probability is clearly negligible: $\varepsilon_{511} = 2^{-lP}$. (2) $\mathcal{A}$ seeks $[X]_2$ or X_i from $[X]_1$, which contradicts SS-Perfect privacy. The probability is negligible as $\varepsilon_{512} = \varepsilon_{SS} - 2^{-lP}$. In summary, the probability of Case 1 occurring is $\varepsilon_{51} = \varepsilon_{511} + \varepsilon_{512} \approx \varepsilon_{SS} \leq \text{negl}(\lambda)$.

Case 2. $\mathcal{A}$ could gain $[X]_2$. (1) $\mathcal{A}$ receives $[X]_2$ from $\mathcal{DS}$ or $\mathcal{S}_2$, which contradicts security assumption. The probability $\varepsilon_{521} = 2^{-lP|\mathcal{U}_1|}$ is negligible. (2) $\mathcal{A}$ seeks $[X]_2$ or X from π_{MatMul} which is an MPC protocol, which contradicts MPC-Input privacy. When performing secure aggregation, the shared feature matrices $[X]_b$ are input into π_{MatMul} . As described in MPC-Input privacy, the probability of getting another input $[X]_2$ is negligible. $\varepsilon_{522} = \varepsilon_{MPC} - 2^{-lP|\mathcal{U}_1|}$. In summary, the probability of Case 2 occurring is $\varepsilon_{52} \approx \varepsilon_{MPC} \leq \text{negl}(\lambda)$.

As a result, we know that the value l is sufficiently large, ε_{SS} and ε_{MPC} are negligible. Then, we have:

$$\text{Prob}[X_i, X \leftarrow \mathcal{A}_\Pi([X]_1, [X]_1)] \approx \varepsilon_{SS} + \varepsilon_{MPC} \leq \text{negl}(\lambda),$$

which proves the feature privacy of our scheme. $\square$

F. Location Privacy Against Multiple Queries

Let $\mathcal{A}$ denote an adversary who aims to acquire to inference $\mathcal{DS}_i$'s location (x_i, y_i) based on masked distances $\{mdist_{i,j}\}_{j \in \mathcal{U}}$ aggregated from multiple queries. $mdist$ is composed of the squared true distance $dist$ and a randomly sampled mask r , $mdist_{i,j} = dist_{i,j}^2 - r_{i,j}$.

Lemma 1: Let the incoming view G contain incoming messages $a_1 \pm r, a_2, \dots, a_k$ where a_1, r are elements of finite additive group A and where r is a uniformly random element of A , independent from all a_i . Denote the distribution of the incoming view G as $\mathbf{D}(G)$. Then

$$\mathbf{D}(G) = \mathbf{D}(G[a_1 \pm r/r])$$

where $G[a_1 \pm r/r]$ denotes the game, where the occurrence of $a_1 \pm r$ has been replaced by r [25].

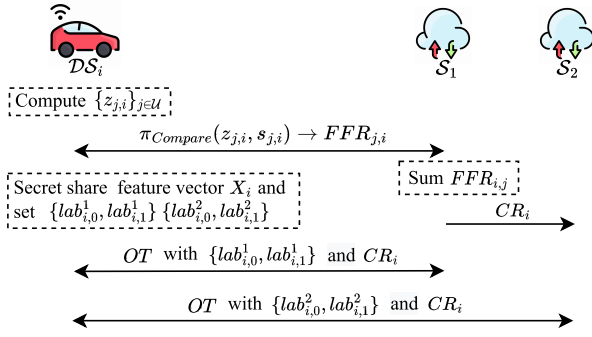


Fig. 6. The process of oblivious data collecting.

Definition 6 (Location privacy against multiple queries) A scheme Π is location privacy against multiple queries if for every PPT adversary $\mathcal{A}$ there is a negligible function $\text{negl}(\cdot)$ such that for a sufficiently large λ ,

$$\text{Prob}[(x_i, y_i) \leftarrow \mathcal{A}_\Pi(\{m\text{dist}_{i,j}\}_{j \in \mathcal{U}'}) : m\text{dist}_{i,j} = \text{dist}_{i,j}^2 - r_{i,j}] \leq \text{negl}(\lambda).$$

Theorem 6: Assuming the distribution of the masked distances satisfies the property in Lemma 1, PEDAs is location privacy against multiple queries.

Proof. When $\mathcal{A}$ outputs location (x_i, y_i) based on $G[m\text{dist}] = \{m\text{dist}_{i,j}\}_{j \in \mathcal{U}'}$, there are two possible cases.

Case 1. $\mathcal{A}$ can inference (x_i, y_i) from multiple masked distances $\{m\text{dist}_{i,j}\}_{j \in \mathcal{U}'}$, where $m\text{dist}_{i,j} = \text{dist}_{i,j}^2 - r_{i,j}$. Thus, it means that the distribution of $G[m\text{dist}]$ remain highly correlated with the true distances $G[\text{dist}] = \{\text{dist}_{i,j}\}_{j \in \mathcal{U}'}$, denoted as $\mathbf{D}(G[m\text{dist}]) = \mathbf{D}(G[\text{dist}])$. To inference (x_i, y_i) based on statistical inference, it is obviously that the distribution of $G[\text{dist}]$ is not equal to a random distribution, denoted $\mathbf{D}(G[\text{dist}]) \neq \mathbf{D}(G[r])$. In this way, we have $\mathbf{D}(G[m\text{dist}]) \neq \mathbf{D}(G[r])$. However, this contradicts Lemma 1. Let ε_r and ε_{61} be the probabilities that Lemma 1 is broken and that $\mathcal{A}$ infers (x_i, y_i) from multiple queries $m\text{dist}_{i,j}\}_{j \in \mathcal{U}'}$, respectively. Therefore, we have $\varepsilon_{61} = \varepsilon_r$.

Case 2. $\mathcal{A}$ could figure out multiple true distances $\{\text{dist}_{i,j}\}_{j \in \mathcal{U}'}$ from masked distances, then inference (x_i, y_i) . However, this contradicts Theorem 2 we have proofed, the probability of $\mathcal{A}$ obtaining distance is a negligible as $\varepsilon_{62} = \varepsilon_2$.

We know that ε_r and ε_2 are negligible. Then, we have:

$$\begin{aligned} \text{Prob}[(x_i, y_i) \leftarrow \mathcal{A}_\Pi(\{m\text{dist}_{i,j}\}_{j \in \mathcal{U}'})] &= \varepsilon_{61} + \varepsilon_{62} \\ &\approx \varepsilon_r + \varepsilon_2 \leq \text{negl}(\lambda), \end{aligned}$$

which proves the location privacy against multiple queries. $\square$

Based on the above security analyses, we can conclude that the proposed PEDAs can achieve such security. In addition, due to space limitations, we omitted details of constructing games.

VI. PERFORMANCE EVALUATION

A. Experimental Settings

We use PBC-0.5.14 [26] to implement the pairing-based cryptographic techniques used in IPFE. We utilize emp-toolkit [27] for OT. The library SCI [14], Cheetah [28] and SecureNN [15] provide us with abundant secure computation modules.

The data we used for the experiment is from a real location-related dataset METR-LA [29] which contains traffic speeds collected from actual loop detectors with precise geographic information. Also, METR-LA is classic and widely used in numerous works [30], [31], [32]. The simulations were performed on an Ubuntu desktop with an Intel Xeon Platinum 8357B CPU @2.70GHz and 128G RAM.

B. Performance Evaluation of Key Components

1) *Private Distances Computation:* The computational costs of $\mathcal{S}$ and $\mathcal{DS}$ are plotted in Fig. 7.

Firstly, we encode the locations of loop detectors in METR-LA to vectors. In this experiment, we directly consider the latitude and longitude of the locations as coordinate points (x, y) . Then we encode to obtain the vector $\vec{CN}$. Each $\mathcal{DS}$ randomly samples random masks for each other $\mathcal{DS}$, and then generates vectors $\vec{NN}$. Assuming the number of data suppliers is n , each $\mathcal{DS}$ needs to perform $n-1$ random value sampling and n encoding to obtain vectors. Therefore, the running time varies linearly with the number of $\mathcal{DS}$ s as shown in Fig. 7a.

We use three types of parameters provided in PBC library for the encryption, which refer to different curves. Below is a brief introduction, more details can be found in the PBC manual. Type A pairings are constructed over F_q with embedding degree $k=2$, where q is 512 bits and group order r is 160 bits. A Type D curve with $k=6$ has order $h*r$ where q is 159 bits, r is prime with 158 bits, and h is a small constant. In Type F, $k=12$, both r and q are roughly 160 bits. According to parameters r , q and k [33], Type F is able to provide the strongest security guarantee among the three. In contrast, Type D and Type A, which are nearly equivalent in terms of security, are relatively weaker.

Fig. 7b and Fig. 7c illustrate the running time of $\mathcal{DS}$ and $\mathcal{S}$ with different numbers of $\mathcal{DS}$ s in the encryption phase and decryption phase respectively. With the extension of the number of $\mathcal{DS}$ s, the running time of each $\mathcal{DS}$ increases linearly. Concerning road distances, there are two distances between each pair $\mathcal{DS}$ s. So, $\mathcal{DS}_i$ generates ciphertexts CenC and NeiD for each $\mathcal{DS}_j$ in Protocol 2. When the number turns to n , $\mathcal{DS}$ needs to execute encryption processes $n-1$ times. As the number of $\mathcal{DS}$ s increases in Fig. 7c, the running time of $\mathcal{S}$ increases quadratically. In Protocol 3, $\mathcal{S}$ computes distances between each pair $\mathcal{DS}$ s. When the number of $\mathcal{DS}$ s increases from $n-1$ to n , the distances that $\mathcal{S}$ needs to calculate increases by $2(n-1)$. Therefore, with the same parameters, the cost of $\mathcal{S}$ increases faster than that of $\mathcal{DS}$.

Fig. 7d shows the detailed time allocation for a set of encryption by $\mathcal{DS}$. In Protocol 2, similar calculations are performed twice related to G_1 and G_2 respectively to generate ciphertexts CenC and NeiD . However, due to different parameter choices, their performance varies between the two groups. Specifically, Type D and Type F have less time overhead on G_1 compared to Type A. In the case of parameter Type D, it has more overhead on G_2 than other parameters. To conclude, Type F performs well on $\mathcal{DS}$ s but incurs significant server overhead, which is suitable for scenarios where weaker computing power on $\mathcal{DS}$ s but stronger computing power on the server. The

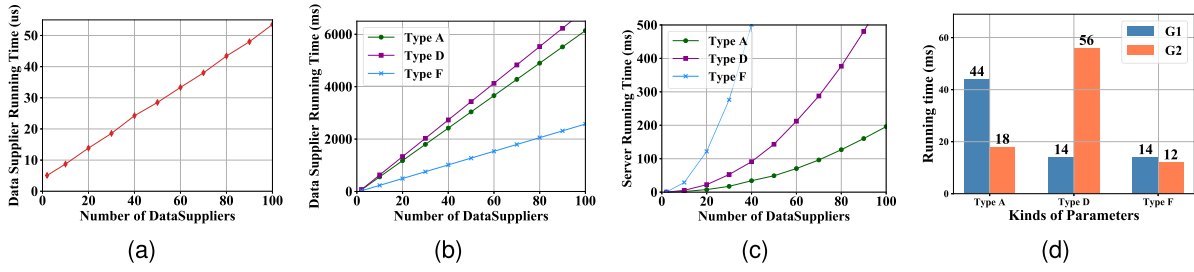


Fig. 7. Computational cost in private distance computation. (a) Data supplier's running time on Positional encoding. (b) Data supplier's running time on encryption phase. (c) Server's running time. (d) Computational cost allocation on encryption phase.

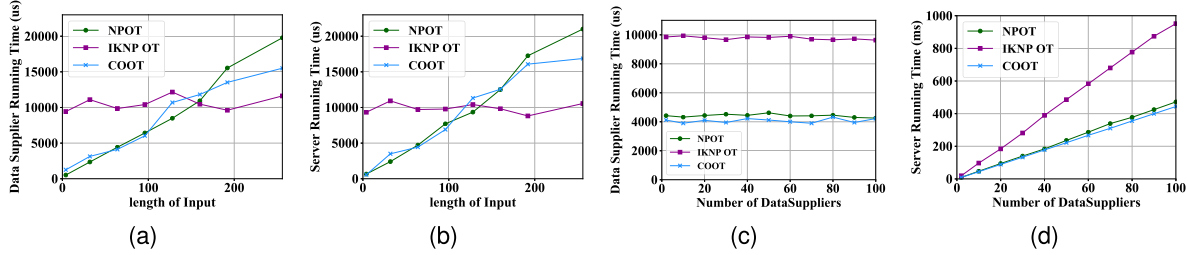


Fig. 8. Computational cost in oblivious data collecting. (a) Data supplier's running time when length increases, (b) Server's running time when length increases, (c) Data supplier's running time when number of DS s increases. (d) Server's running time when the number of DS s increases.

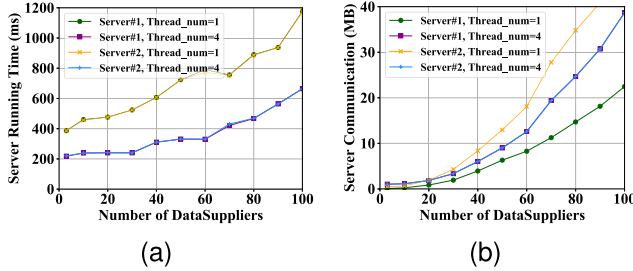


Fig. 9. Computation and Communication cost in privacy-preserving adjacency matrix generation. (a) Server's running time. (b) The bytes sent by the server.

overall time cost of parameter Type A is relatively low, making it suitable for more balanced scenarios.

2) *Oblivious Data Collecting*: We choose three types of OT implementations to experimentally evaluate their performance. Naor–Pinkas OT (NPOT) [34] is a classic discrete-logarithm-based protocol, mainly used for small numbers of OTs or as base OTs. IKNP OT Extension [35] efficiently generates large-scale OTs from a few base OTs using symmetric-key operations. Chou–Orlandi OT (COOT) [36] is a compact Diffie–Hellman-based protocol with low communication overhead, suitable for medium-scale applications.

From Fig. 8a and Fig. 8b, the running time cost varies as the input length changes. The time cost of NPOT and COOT increases with the increase in input length. Alternatively, IKNP OT fluctuates at a fixed value. When the input length is around 160 bits, they have the closest time cost. When the input length changes, the performance shown in Fig. 8a and Fig. 8b are almost consistent. But when the number of DS s changes, Fig. 8c and Fig. 8d are completely different. The running time of S varies linearly with the number of DS s. This is because S needs to interact with each DS to collect features. On the contrary, DS only needs to interact with S , and DS s' running time is independent of the number of DS s.

3) *Privacy Preserving Adjacency Matrix Generation*: Fig. 9 shows the computation and communication costs of $\pi_{S_{qrt}}$ when generating an adjacency matrix. From Fig. 9a, it can be seen that S_1 and S_2 have basically the same running time, regardless of the number of threads. As the number of DS s increases, the overall running time of S_1 and S_2 shows an increasing tendency. When the thread is increased from 1 to 4, the running time decreases, and its speed becomes a little smoother as the number of DS s increases. Fig. 9b illustrates the communication overhead. More specifically, when the number of threads is set to 1, one server (S_2 in the figure) bears the majority of communication overhead while the other (S_1) is relatively less. However, the communication overhead allocation is relatively average concerning the number of threads is 4. When the number of DS s is 20, the communication cost of S_2 is almost identical.

C. Performance Evaluation Under Multiple Dimensions

1) *Node Scalability Evaluation*: To quantitatively evaluate the scalability of PEDA's computational overhead as the number of data suppliers increases, we conducted experiments measuring the costs incurred by data suppliers DS , server S_1 , and server S_2 . In these experiments, we configured $agg(\cdot) = AX$ and $upd(\cdot) = ReLU$, and implemented the system using the SecureNN library. We assume the number of data suppliers is n , with $X \in \mathbb{R}^{n \times 1000}$ and $A \in \mathbb{R}^{n \times n}$ and the distance metric is symmetric. Additional cryptographic parameters include Type A pairing and IKNP OT.

To address the substantial overhead observed in large-scale deployments, we propose an optimization strategy based on spatial partitioning. This strategy groups DS s into K disjoint, coarse-grained spatial regions (e.g., $K = 5$ for 500 DS s) according to their geographical distribution. Prior to local vector encryption, each DS transmits only its region label (e.g., city district or ZIP code zone) to S_1 . Subsequently, the

TABLE III
RUNTIME IN PEDA UNDER VARYING NODE SCALES (S)

	n=50	n=100	n=200	n=500	n=1000
$\mathcal{DS}$	1.59	3.14	6.24	3.15	3.14
$\mathcal{S}_1$	1.63	3.19	7.49	16.38	34.84
$\mathcal{S}_2$	1.03	1.95	4.58	10.18	22.45

TABLE IV
GENERALIZATION OF PEDA ACROSS MULTIPLE APPLICATION DOMAINS

Scenario	Parameters Nodes / Features	Runtime (s)		
		$\mathcal{DS}$	$\mathcal{S}_1$	$\mathcal{S}_2$
Traffic	207 / 34272	6.43	15.66	12.79
Healthcare	700 / 5000	5.80	30.34	25.40
Social networks	1502 / 2071	6.03	66.78	52.23

servers perform distance computation, feature matrix generation, and adjacency matrix construction independently within each region, after which the partitioned results are merged into unified global structures for final aggregation.

The computational costs under varying numbers of data suppliers are summarized in TABLE III. This optimization discloses only coarse-grained region membership rather than precise coordinates, thereby ensuring that PEDA's core privacy objectives including location privacy, distance privacy, adjacency matrix privacy, collecting strategy privacy, and feature privacy, remain strictly preserved. By intentionally limiting the granularity of spatial information, the proposed optimization achieves a significant reduction in computational overhead while maintaining privacy guarantees.

2) *Generalization in Sensitive Scenarios*: To further validate the generalization of PEDA, we extend the experiments to two additional sensitive scenarios: social networks and healthcare. The protocol remains unchanged: nodes are mapped to private two-dimensional coordinates. The semantics of the coordinates vary by domain: (i) road sensors use geographical locations (traffic); (ii) users embed trust values (social); (iii) patients are represented by clinical embeddings (healthcare). In all cases, the adjacency matrix is then constructed based on pairwise similarity (distance) and features are collected obliviously.

Table IV reports runtime across the three scenarios. As expected, the cost mainly scales with the number of nodes and feature dimensionality. The social network case (TrustFilm [37] with 1502 users and 2071 features) yields the highest runtime, while healthcare (simulated with 700 patients and 5000 features) with fewer nodes shows moderate overhead. These results demonstrate that our scheme generalizes beyond location-related scenarios like traffic and efficiently supports sensitive domains such as social networks and healthcare.

3) *Impact of Network Latency*: To evaluate the impact of network latency on PEDA, we benchmarked its runtime under two representative settings: a LAN environment (3Gbps bandwidth, 0.8ms latency) and a WAN environment (200Mbps bandwidth, 40ms latency). As shown in Fig. 10, we measured the runtime of PEDA with 20, 50, and 100 data suppliers. Results show that network latency has a pronounced impact due to the reliance on multi-round MPC protocols. For instance, at 100 data suppliers, the oblivious data collecting

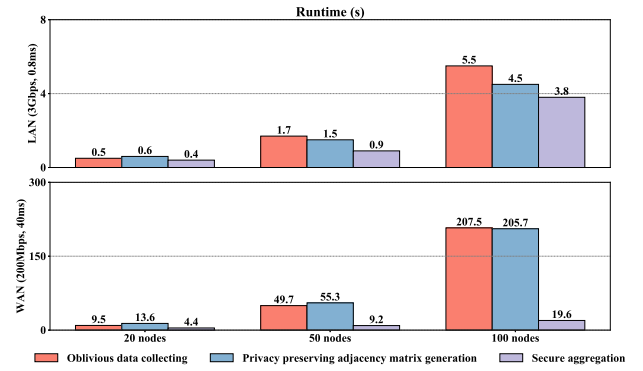


Fig. 10. Impact of network latency.

TABLE V
COMPARISON OF RUNTIME WITH AND WITHOUT PEDA

	Runtime (mins, Single)	Increase (Single)	Avg.Runtime (mins, N=10)	Increase (N=10)
General PPML [15]	0.92		0.92	
PEDA	2.36	2.57 ×	1.06	1.15 ×
SecGNN [38]	15.93		15.93	
SecGNN+PEDA	17.53	1.10 ×	16.89	1.06 ×
CoGNN [39]	38.73		38.73	
CoGNN+PEDA	40.33	1.04 ×	38.89	1.00 ×

and adjacency matrix generation phases take approximately 5.5s and 4.5s, respectively. In contrast, the execution time under the WAN setting increases obviously. These findings indicate that PEDA runs efficiently in low-latency networks but incurs noticeable overhead in high-latency WAN scenarios, suggesting that future work could focus on reducing interactive rounds or using more communication efficient MPC primitives.

4) *Comparison With Related Works*: We have noticed that some secret-share based methods such as SecGNN [38] and CoGNN [39] are used to provide privacy guarantees in GNNs. However, they merely ensure privacy during training and inference, ignoring the privacy risks inherent in generating the adjacency/feature matrices and the data collection phase. In comparison to these works, a key advantage of our solution is its ability to mitigate privacy risks across all stages, effectively addressing the limitation mentioned above.

Table V summarizes the runtime comparison between PEDA and representative frameworks. In this experiment, there are 1708 nodes with 1433 features. For the aggregation based on general PPML [15], PEDA introduces an additional cost of a 2.57× increase for a single training run. However, this cost is a one-time preprocessing overhead and can be amortized across multiple runs, denoted by N . For instance, the average runtime (Avg.Runtime) increase drops to merely 1.15× when $N = 10$, i.e., less than a few seconds per run. Similarly, when integrated with large-scale models such as SecGNN [38] and CoGNN [39]. Following the original papers, we train CoGNN for 90 epochs and SecGNN for 30 epochs. PEDA only incurs marginal overheads: 1.10× and 1.04×, respectively. After amortization across multiple runs, the overhead is further reduced to 1.06× for SecGNN and nearly negligible (1.00×) for CoGNN. In particular, using the first three modules, PEDA

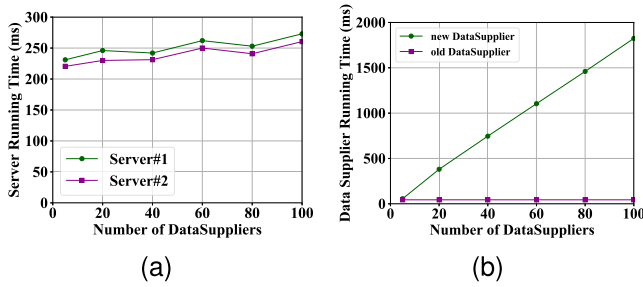


Fig. 11. Computational cost when new node joining. (a) Server's running time. (b) Data supplier's running time.

TABLE VI
COMPARISON WITH RELATED WORKS

	Classic GNN [40]	General PPML [15]	Graph- AT [41]	Solitude [42]	FedGNN [43]	SecGNN [38]	PEDA
G_1	✓	×	×	×	×	×	✓
G_2	×	×	×	×	×	×	✓
G_3	×	×	×	×	×	×	✓
G_4	×	×	×	✓	✓	✓	✓
G_5	×	✓	×	✓	✓	✓	✓
G_6	✓	✓	✓	×	✓	✓	✓
G_7	×	×	×	×	×	×	✓

G_1 : Distance-aware. G_2 : Location privacy. G_3 : Distance privacy.
 G_4 : Adjacency matrix privacy. G_5 : Feature privacy.
 G_6 : Maintaining high data utility. G_7 : Oblivious data collecting.

generates the adjacency matrix and feature matrix without leaking extra information about data suppliers, thereby filling an important gap left by existing schemes.

These results demonstrate that the runtime overhead of PEDA is nearly negligible compared with the overall time, especially in realistic experimental settings where the data is reused multiple times. Importantly, this minor cost enables the secure generation of adjacency and feature matrices without exposing the raw data of suppliers, thereby making PEDA a practical and lightweight enhancement.

5) *Evaluation in Dynamic Scenarios*: We evaluate the computational overhead associated with the scenario of a single newly joining node under the condition that the pairwise distance metric is symmetric, i.e., $dist_{i,j} = dist_{j,i}$. Fig. 11 shows that the server's running time increases with the number of existing data suppliers, ranging from approximately 220 ms to 270 ms as the number of nodes increases from 20 to 100. For data suppliers, the newly joining node experiences a linear increase in runtime due to distance computations with all existing nodes, whereas existing nodes incur only minimal and constant overhead, regardless of the number of nodes. Thus, these results demonstrate that the proposed approach introduces only localized costs for node addition, thereby confirming its adaptability in dynamic scenarios.

VII. RELATED WORK

We have summarized the comparisons with related works in TABLE VI, each of which can represent a type of work.

The first type is machine learning algorithms that are suitable for location-related scenarios but do not consider privacy risks at all. For example, Zhao et al. [40] propose a traffic forecasting method, which is a classic GNN model that

is distance-dependent but does not support any privacy guarantees. Unlike this, the second type is popular privacy-preserving machine learning algorithms but lacks consideration for the characteristics of spatial data. General PPML [15] design a secure computation for neural networks, which only considers feature privacy, instead of characteristics of graph.

The existing work on privacy-preserving graph neural networks (PPGNN) has limitations. The most common methods are based on differential privacy (DP), federated learning (FL) and secret sharing (SS). Solitude [42] combine DP which adds noise into both node features and graph structure, achieving acceptable performance while preventing data breaches [44]. Additionally, due to DP adds noise to features, such combination [45] compromise data utility and give no method to support distance awareness and location privacy. Moreover, some FL-based approaches [46], [47] such as FedGNN [43], in which clients train features locally while the server aggregates the models, may keep sensitive features local. However, such methods are often not suitable for scenarios where the entire graph needs to be manipulated [8]. Moreover, FL allocates the cost of model training to data suppliers, which have certain requirements for their computing abilities. Furthermore, some works like SecGNN [39] and CoGNN [38] split adjacency matrix and feature matrix using secret sharing to ensure privacy. However, such PPGNN works do not consider the privacy risks related to matrix generation.

Furthermore, Li et al. [41] give a graph adversarial training (Graph-AT) framework that preserves the structural information and features. Similarly, Liao et al. [48] utilize adversarial training to locally filter out pre-determined sensitive attributes. However, they focus on privacy against inference attacks where the adversary is usually an external attacker, while we study private learning to resist internal attackers. Therefore, they can not achieve the properties we assumed.

Compared to them, our method maintains the features' usability and is suitable for the entire graph. Moreover, we focus on location privacy and achieve distance awareness. In addition, we also address privacy issues in the generation of the adjacency matrix and collecting features.

VIII. DISCUSSION

A. Stronger Security

An important direction for strengthening the security of our protocol is to move towards the malicious model. A natural way is to adopt the SPDZ authenticated secret sharing [49]. In this approach, every additive share is augmented with a share of a global MAC, which allows parties to verify the correctness of openings and thereby detect any attempt to forge shares. Incorporating such techniques into our protocol would prevent malicious deviations such as forging inconsistent shares and significantly raise the security guarantees. We believe that many components of our scheme can be upgraded to malicious security directly using SPDZ sharing. But, our construction also involves more complex primitives such as secure square root. Although these operations are supported in the MP-SPDZ framework, integrating them efficiently into our protocol while preserving security guarantees requires further

investigation. We therefore view a systematic extension of our framework with SPDZ sharing against malicious adversaries as an important direction for future work.

While the proposed framework effectively protects raw features during computation, feature privacy may still be compromised via attacks targeting the deployed model itself, such as Model Inversion Attacks. To mitigate this, we plan to integrate Differential Privacy by injecting calibrated noise into outputs. Such DP-enhanced approach provides rigorous, provable guarantees against feature reconstruction, ensuring comprehensive protection for features both during computation and in the deployed model [50].

B. More Efficient Method

Our current framework relies on MPC, which inevitably requires frequent interactions among participating parties and results in relatively high communication overhead. Although we have introduced a spatial partitioning strategy to mitigate these challenges and enable scalability to larger node populations, there remain opportunities for further improvement. In future work, we plan to investigate more communication-efficient MPC protocols that incorporate lightweight cryptographic primitives or hybrid secure computation techniques to reduce the cost of interaction. We also aim to investigate additional optimizations in the partitioning process to enhance overall computational efficiency. Furthermore, we intend to extend our evaluation to more complex and larger-scale scenarios for further improvement in practical deployments.

IX. CONCLUSION

In this paper, we propose a data aggregation framework ensuring privacy-preservation and distance awareness in GNNs. We put forward a new distance computation method based on functional encryption for computing distances between encrypted locations. Also, a new adjacency matrix generation measure is designed to implicitly and privately capture correlations between nodes. Likewise, we develop oblivious data collecting to improve the usability of collected features and avoid leaking any spatial information. We carefully analyze and evaluate the proposed protocols. The results show that our proposed scheme has more comprehensive security considerations and usability at an acceptable time cost.

Future work will focus on improving efficiency beyond the limitations of the MPC-based design and strengthening security against malicious and collusive adversaries.

REFERENCES

- [1] Y. Wang, Y. Zhao, Y. Zhang, and T. Derr, "Collaboration-aware graph convolutional network for recommender systems," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 91–101.
- [2] M. Réau, N. Renaud, L. C. Xue, and A. M. J. J. Bonvin, "DeepRank-GNN: A graph neural network framework to learn patterns in protein-protein interfaces," *Bioinformatics*, vol. 39, no. 1, p. 759, Jan. 2023.
- [3] W. Kong, Z. Guo, and Y. Liu, "Spatio-temporal pivotal graph neural networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 8, pp. 8627–8635.
- [4] B. Wu, W. Chen, W. Wang, B. Peng, L. Sun, and L. Chen, "Weathergnn: Exploiting meteo- and spatial-dependencies for local numerical weather prediction bias-correction," in *Proc. Int. Joint Conf. Artif. Intell.*, Mar. 2024, pp. 2433–2441.
- [5] J. Simeunović, B. Schubnel, P.-J. Alet, and R. E. Carrillo, "Spatio-temporal graph neural networks for multi-site PV power forecasting," *IEEE Trans. Sustain. Energy*, vol. 13, no. 2, pp. 1210–1220, Apr. 2022.
- [6] S. Sajadmanesh, A. S. Shamsabadi, A. Bellet, and D. Gática-Pérez, "GAP: Differentially private graph neural networks with aggregation perturbation," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, Aug. 2022, pp. 3223–3240.
- [7] G. Mei, Z. Guo, S. Liu, and L. Pan, "SGNN: A graph neural network based federated learning approach by hiding structure," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2019, pp. 2560–2568.
- [8] Y. Zhang et al., "A survey on privacy in graph neural networks: Attacks, preservation, and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 7497–7515, Dec. 2024.
- [9] P. de Perthuis and D. Pointcheval, "Two-client inner-product functional encryption with an application to money-laundering detection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 725–737.
- [10] A. Beimel, "Secret-sharing schemes: A survey," in *Proc. Int. Conf. Coding Cryptol.*, 2011, pp. 11–46.
- [11] M. O. Rabin, "How to exchange secrets with oblivious transfer," *Cryptol. ePrint Arch.*, Paper 2005/187, 2005[Online]. Available: <https://eprint.iacr.org/2005/187>
- [12] M. Naor and B. Pinkas, "Oblivious transfer and polynomial evaluation," in *Proc. thirty-first Annu. ACM Symp. Theory Comput.*, May 1999, pp. 245–254.
- [13] D. Rathee et al., "CrypTFlow2: Practical 2-party secure inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 325–342.
- [14] D. Rathee, A. Bhattacharya, R. Sharma, D. Gupta, N. Chandran, and A. Rastogi, "SecFloat: Accurate floating-point meets secure 2-party computation," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 576–595.
- [15] S. Wagh, D. Gupta, and N. Chandran, "SecureNN: 3-party secure computation for neural network training," *Proc. Privacy Enhancing Technol.*, vol. 2019, no. 3, pp. 26–49, Jul. 2019.
- [16] D. Hurtado Ramírez and J. M. Auñón, "Privacy preserving K-means clustering: A secure multi-party computation approach," 2020, *arXiv:2009.10453*.
- [17] A. C. Yao, "Protocols for secure computations," in *Proc. 23rd Annu. Symp. Found. Comput. Sci. (SFCS)*, Nov. 1982, pp. 160–164.
- [18] H. Ren et al., "Graph convolutional networks in language and vision: A survey," *Knowl.-Based Syst.*, vol. 251, Sep. 2022, Art. no. 109250.
- [19] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016[Online]. Available: <https://openreview.net/forum?id=SJU4ayYgl>
- [20] B. Wang, M. Li, H. Wang, and H. Li, "Circular range search on encrypted spatial data," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Sep. 2015, pp. 182–190.
- [21] Z. Cui, K. Henrickson, R. Ke, and Y. Wang, "Traffic graph convolutional recurrent neural network: A deep learning framework for network-scale traffic learning and forecasting," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 11, pp. 4883–4894, Nov. 2020.
- [22] F. D. Protection, "General data protection regulation (GDPR)," *Intersoft Consulting*, vol. 24, no. 1, pp. 1–5, 2018.
- [23] J. Ye, J. Zhao, K. Ye, and C. Xu, "How to build a graph-based deep learning architecture in traffic domain: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 5, pp. 3904–3924, May 2022.
- [24] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, "The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains," *IEEE Signal Process. Mag.*, vol. 30, no. 3, pp. 83–98, May 2013.
- [25] D. Bogdanov, M. Nitssoo, T. Toft, and J. Willemson, "High-performance secure multi-party computation for data mining applications," *Int. J. Inf. Secur.*, vol. 11, no. 6, pp. 403–418, Nov. 2012.
- [26] B. Lynn, "The pairing-based cryptography (PBC) library," Stanford Univ., Stanford, CA, USA, Tech. Rep., 2010.
- [27] X. Wang, A. J. Malozemoff, and J. Katz, "EMP-toolkit: Efficient multiparty computation toolkit," Univ. Maryland, College Park, MD, USA, Tech. Rep., 2016. [Online]. Available: <https://github.com/emp-toolkit>
- [28] Z. Huang, W.-J. Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure two-party deep neural network inference," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur.)*, Mar. 2022, pp. 809–826.

- [29] H. V. Jagadish et al., "Big data and its technical challenges," *Commun. ACM*, vol. 57, no. 7, pp. 86–94, 2014.
- [30] R. Jiang et al., "Spatio-temporal meta-graph learning for traffic forecasting," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2023, vol. 37, no. 7, pp. 8078–8086.
- [31] Z. Lai, D. Zhang, H. Li, C. S. Jensen, H. Lu, and Y. Zhao, "LightCTS: A lightweight framework for correlated time series forecasting," *Proc. ACM Manage. Data*, vol. 1, no. 2, pp. 1–26, Jun. 2023.
- [32] I. Marisca, A. Cini, and C. Alippi, "Learning to reconstruct missing data from spatiotemporal graphs with sparse observations," in *Proc. 36th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 1–17.
- [33] A. Menezes, S. Vanstone, and T. Okamoto, "Reducing elliptic curve logarithms to logarithms in a finite field," in *Proc. 23rd Annu. ACM Symp. Theory Comput. (STOC)*, 1991, pp. 80–89.
- [34] M. Naor and B. Pinkas, "Efficient oblivious transfer protocols," in *Proc. 12th Annu. ACM-SIAM Symp. Discrete Algorithms*, 2001, pp. 448–457.
- [35] Y. Ishai, J. Kilian, K. Nissim, and E. Petrank, "Extending oblivious transfers efficiently," in *Proc. Annu. Int. Cryptol. Conf.*, vol. 2729, Santa Barbara, CA, USA, Aug. 2003, pp. 145–161.
- [36] T. Chou and C. Orlandi, "The simplest protocol for oblivious transfer," in *Proc. 4th Int. Conf. Cryptol. Inf. Secur. Latin Amer.*, 2015, pp. 40–58.
- [37] G. Guo, J. Zhang, and N. Yorke-Smith, "A novel Bayesian similarity measure for recommender systems," in *Proc. 23rd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2013, pp. 2619–2625.
- [38] S. Wang, Y. Zheng, and X. Jia, "SecGNN: Privacy-preserving graph neural network training and inference as a cloud service," *IEEE Trans. Services Comput.*, vol. 16, no. 4, pp. 2923–2938, Jul. 2023.
- [39] Z. Zou, Z. Liu, J. Shan, Q. Li, K. Xu, and M. Xu, "CoGNN: Towards secure and efficient collaborative graph learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 4032–4046.
- [40] L. Zhao et al., "T-GCN: A temporal graph convolutional network for traffic prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 9, pp. 3848–3858, Sep. 2020.
- [41] K. Li, G. Luo, Y. Ye, W. Li, S. Ji, and Z. Cai, "Adversarial privacy-preserving graph embedding against inference attack," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6904–6915, Apr. 2021.
- [42] W. Lin, B. Li, and C. Wang, "Towards private learning on decentralized graphs with local differential privacy," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2936–2946, 2022.
- [43] C. Wu, F. Wu, Y. Cao, Y. Huang, and X. Xie, "FedGNN: Federated graph neural network for privacy-preserving recommendation," 2021, *arXiv:2102.04925*.
- [44] F. Wu, Y. Long, C. Zhang, and B. Li, "LINKTELLER: Recovering private edges from graph neural networks via influence analysis," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 2005–2024.
- [45] Z. Xiang, T. Wang, and D. Wang, "Preserving node-level privacy in graph neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 4714–4732.
- [46] C. Chen et al., "Vertically federated graph neural network for privacy-preserving node classification," in *Proc. Int. Joint Conf. Artif. Intell. Org. (IJCAI)*, 2022, pp. 1959–1965.
- [47] Z. Han, C. Hu, T. Li, Q. Qi, P. Tang, and S. Guo, "Subgraph-level federated graph neural network for privacy-preserving recommendation with meta-learning," *Neural Netw.*, vol. 179, Mar. 2024, Art. no. 106574.
- [48] P. Liao et al., "Information obfuscation of graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, Apr. 2020, pp. 6600–6610.
- [49] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Proc. Annu. Cryptol. Conf.*, Apr. 2012, pp. 643–662.
- [50] M. Fredrikson, E. Lantz, S. Jha, S. Lin, D. Page, and T. Ristenpart, "Privacy in pharmacogenetics: An end-to-end case study of personalized warfarin dosing," in *Proc. 23rd USENIX Secur. Symp. (USENIX Secur.)*, Mar. 2014, pp. 17–32.



Junwei Zhang received the B.S., M.S., and Ph.D. degrees in computer science and technology from Xidian University, China, in 2004, 2006, and 2010, respectively. He is currently a Professor with the School of Cyber Engineering, Xidian University. His research interests include cryptography and information security, especially in provable security and data location security in cloud computing.



Zhuo Ma (Senior Member, IEEE) received the Ph.D. degree in computer architecture from Xidian University, Xi'an, China, in 2010. He is currently a Professor with the School of Cyber Engineering, Xidian University. His research interests include cryptography, machine learning in cyber security, and the Internet of Things security.



Jinhai Zhang received the B.S. degree in information security from Xidian University, China, in 2020, where he is currently pursuing the Ph.D. degree with the School of Cyber Engineering. His research interests include multi-party computation and privacy-preserving machine learning protocols.



Xinghua Li (Member, IEEE) received the M.S. and Ph.D. degrees in computer science from Xidian University, China, in 2004 and 2007, respectively. He is currently a Professor with the School of Cyber Engineering, Xidian University. His research interests include wireless network security, privacy protection, cloud computing, and security protocol formal methodology.



Yuqing Wang received the B.S. degree in information security from Xidian University in 2020, where she is currently pursuing the Ph.D. degree with the School of Cyber Engineering. Her research interests include privacy preservation, data security, secure computation, and cryptography.



Jianfeng Ma (Member, IEEE) received the Ph.D. degree from Xidian University, Xi'an, China, in 1995. He has been a Professor with the Department of Computer Science and Technology, Xidian University, since 1998. He was a special engaged Professor of the Yangtze River Scholar, China. His research interests include cryptology, network security, and data security.